

GSP-KalmanNet: Tracking Graph Signals via Neural-Aided Kalman Filtering

Itay Buchnik , Guy Sagi , Nimrod Leinwand , Yuval Loya , Nir Shlezinger , *Senior Member, IEEE*,
and Tirza Routtenberg , *Senior Member, IEEE*

Abstract—Dynamic systems of graph signals are encountered in various applications, including social networks, power grids, and transportation. While such systems can often be described as state space (SS) models, tracking graph signals via conventional tools based on the Kalman filter (KF) and its variants is typically challenging. This is due to the nonlinearity, high dimensionality, irregularity of the domain, and complex modeling associated with real-world dynamic systems of graph signals. In this work, we study the tracking of graph signals using a hybrid model-based/data-driven approach. We develop the *GSP-KalmanNet*, which tracks the hidden graphical states from the graphical measurements by jointly leveraging graph signal processing (GSP) tools and deep learning (DL) techniques. The derivations of the *GSP-KalmanNet* are based on extending the KF to exploit the inherent graph structure via designing a graph frequency domain filtering and replacing the Kalman gain (KG) with a graph filter that minimizes the prediction error. Thus, it considerably simplifies the computational complexity entailed in processing high-dimensional signals and increases the robustness to small topology changes. Then, we use data to learn the KG, namely, the graph filter, following the recently proposed KalmanNet framework, which copes with partial and approximated modeling, without forcing a specific model over the noise statistics. Restricting the KG to a graph filter in the proposed *GSP-KalmanNet* reduces learned parameters, thereby enhancing stability. Our empirical results demonstrate that the *GSP-KalmanNet* achieves enhanced accuracy and run time performance, and improved robustness to model misspecifications compared with both model-based and data-driven benchmarks.

Index Terms—Graph signal processing (GSP), graph filters, deep learning, Kalman filtering.

I. INTRODUCTION

GRAPH signals are a fundamental model for describing multivariate signals of irregular and complex structures associated with networks [2], [3]. These include, for example, transportation systems, social networks, sensor deployments, and power systems. Various applications involve tasks represented as the tracking time sequences of graph signals from nonlinear graph signal observations, representing measurements taken over the network, e.g., in power grids [4], [5], [6].

The Kalman filter (KF) [7] is the cornerstone algorithm for tracking in dynamic systems. The KF relies on the ability to represent the system as a linear Gaussian state space (SS) model, while its variants, such as the extended Kalman filter (EKF) [8] and unscented Kalman filter (UKF) [9], can track in nonlinear dynamics [10]. However, KF-based tracking of graph signals in nonlinear SS models is subject to several core limitations. First, the high dimensionality of graph signals significantly increases the computational complexity and latency, and also affects the stability of tracking algorithms. Moreover, SS models used in applications involving tracking of graph signals, such as monitoring of power networks [11], [12], [13], are often approximations of complex underlying dynamics. Such mismatches can significantly degrade KF-based tracking.

The complexity associated with the high dimensionality of graph signals is typically tackled (though not necessarily in the context of tracking) by exploiting structures in the data arising from the graph/network. A common framework to do so is graph signal processing (GSP), which extends concepts from traditional digital signal processing for analyzing, representing, and processing data indexed on graphs and signals on networks [2]. GSP leverages the underlying graph structure to capture relationships between data points, while the signals themselves carry meaningful information associated with each node. This framework introduces tools such as the graph Fourier transform (GFT), graph filters, and methods for the sampling and recovery of graph signals [14], [15], [16], [17]. These tools facilitate the exploitation of the graph structure to propose low-complexity estimators via graph filters that are robust to graph topology changes and mismatches [18], [19], [20], [21], [22].

The above estimators consider a single graph signal rather than a time sequence of graph signals. In principle, one can recast a time sequence of graph signals over a fixed time horizon as a spatio-temporal graph, incorporating both temporal and

Manuscript received 13 October 2023; revised 6 May 2024; accepted 22 July 2024. Date of publication 9 August 2024; date of current version 20 August 2024. This work was supported in part by Israeli Ministry of National Infrastructure, Energy, and Water Resources and in part by Israel Science Foundation under Grant 1148/22. An earlier version of this paper was presented in part at the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP) [DOI: 10.1109/ICASSP49357.2023.10096261]. The associate editor coordinating the review of this article and approving it for publication was Dr. Feng Yin. (*Corresponding author: Itay Buchnik.*)

The authors are with the School of Electrical and Computer Engineering (ECE), Ben-Gurion University of the Negev, Be'er Sheva 84105, Israel (e-mail: Itaybuch@post.bgu.ac.il; sagix@post.bgu.ac.il; nimrod@post.bgu.ac.il; loyay@post.bgu.ac.il; nirshl@bgu.ac.il; tirzar@bgu.ac.il).

Digital Object Identifier 10.1109/TSP.2024.3435935

spatial structures in its graph. As this approach considers the joint processing of a given time frame, it is less suitable for real-time tracking, where the goal is to dynamically estimate each incoming observation, as in KF-based tracking. The KF has been employed in several graph-related settings, such as distributed state estimation over sensor networks [23], [24], tracking network traffic dynamics [25], graph-time filters [26], tracking bandlimited graph signals [27], [28], [29], and sampling strategies [30]. However, these techniques are model-based, i.e., they rely on accurate knowledge of the SS model, and thus share the sensitivity of the classical KF to mismatches and approximation errors. Recent works discuss the KF for the evaluation of time-series data that is obtained from a graphical linear Gaussian SS model with unknown topology. In this context, Bayesian approaches for estimating the state transition matrix and the noise covariance have been proposed in [31], [32]. An expectation-maximization algorithm for estimating both the transition matrix and the states has been proposed in [33]. However, in these works, the tracking is of the graph topology, which is assumed to be stationary. To conclude, a model-based Kalman filter for nonlinear graphical processes that tackles mismatches and unknown models does not exist.

An alternative tracking approach, which does not rely on SS models, learns the scheme from data via machine learning. The growing popularity of deep neural networks (DNNs) gave rise to various architectures, such as recurrent neural networks (RNNs) [34] and attention mechanisms [35], that are suitable for processing time sequences. Those architectures used to design DNNs for tracking, inspired by model-based filtering algorithms [36], [37], [38], [39], [40], [41]. A candidate family of DNN architectures for processing graph signals is graph neural networks (GNNs) [42], [43], [44], [45], which incorporate the graph structure into their learned operation. GNNs can also be combined with RNNs for processing time-varying graph signals [46], [47]. Nevertheless, data-driven methods, including GNN-based ones, often suffer from difficulties in training, high computational cost, and generalization issues. Moreover, they do not exploit available (though possibly mismatched) knowledge regarding the SS model, and operate in a black-box manner, lacking the interpretability of model-based tracking.

An emerging paradigm for combining model-based signal processing with data-driven approaches is that of model-based deep learning (DL) [48], [49], [50], [51], [52]. Model-based DL encompasses a set of methodologies for fusing statistical models with DL to enhance interpretability, reduce the need for extensive labeled data, and provide reliable predictions [51]. In the context of tracking in dynamic systems, the KalmanNet algorithm proposed in [53] augments the EKF with a dedicated RNN, which enables learning to track in partially known SS models while preserving the SS-aware EKF. While its hybrid model-based/data-driven design was shown to improve accuracy under mismatched and nonlinear models [53], reduce latency [54], provide confidence measures [55], [56], and facilitate learning [57], its complexity significantly increases when processing high-dimensional signals. Existing extensions for mitigating complexity do so by mapping the observations into a low-dimensional subspace [58], thus not

handling high-dimensional state settings, as in tracking graph signals in large networks. This motivates accounting for the graph structure in model-based/data-driven tracking of graph signals.

In this paper, we propose *GSP-KalmanNet*, a hybrid data-driven/model-based algorithm for real-time low-complexity tracking of graph signals. GSP-KalmanNet is based on the integration of GSP properties into the KalmanNet methodology by identifying the suitable graph filter for handling the complexity and nonlinearity of the SS model, while augmenting its operation with a dedicated RNN to cope with mismatches and unknown stochasticity. The resulting GSP-KalmanNet is suitable for tracking graph signals in a scalable and low-complexity manner, and learns to operate in possibly mismatched and approximated dynamic modeling.

Specifically, we first assume that the dynamic system of graph signals can be described as a known nonlinear SS model, where the main challenge stems from its high dimensionality and nonlinearity. For such settings, we develop GSP-EKF, which alleviates the complexity of the EKF by tracking in the graph frequency domain, while taking the Kalman gain (KG) to be a graph filter. We analyze the GSP-EKF, showcasing the complexity reduction of its KG computation, and characterize analytical conditions for it to achieve the EKF performance. By identifying GSP-EKF as the suitable model-based algorithm in the presence of full domain knowledge, we follow the KalmanNet methodology and augment it into GSP-KalmanNet, addressing partial knowledge of the SS model. By identifying the graph filter KG of the GSP-EKF as its specific computation encapsulating the missing domain knowledge, we augment it with a dedicated RNN that preserves the scalable graph filter operation. We propose a learning algorithm that treats the overall tracking algorithm as a discriminative machine learning architecture [59], training the internal RNN of GSP-KalmanNet to produce a graph filter that minimizes the mean-squared error (MSE) in the GSP-EKF overall flow.

Our main contributions are summarized as follows:

- **Identifying the Core Challenges of Tracking Graph Signals:** Our derivation is based on a principled formulation of the inherent challenges associated with tracking graph signals. These include the inherent complexity when processing high-dimensional graph signals, the likely presence of mismatches in the graph topology or SS modeling, and the expected unknown stochasticity.
- **GFT-Based Processing as Key for Alleviating Complexity in Tracking:** We rigorously pinpoint the usage of graph structure via processing in the GFT domain as a means for alleviating the complexity and scalability issues that arise when processing graph signals.
- **GSP-Informed DNN-Augmented Algorithm:** We leverage the above contributions to propose GSP-KalmanNet algorithm. Our novel algorithm is based on a principled combination of a set of key signal processing techniques. It combines GSP tools to alleviate complexity by exploiting spatial structure, with SS model-based methods to leverage temporal domain knowledge, integrated

with deep learning into a novel hybrid model-based/data-driven algorithm.

- **Extensive Numerical Evaluations:** Through extensive numerical experiments, we demonstrate the gains of the proposed methodology for synthetic simulations and for power system state estimation (PSSE). We systematically show performance and robustness gains of each of the ingredients of GSP-KalmanNet compared to both model-based and data-driven benchmarks, as well as demonstrate operational gains associated with the hybrid model-based/data-driven design in terms of sample complexity and uncertainty extraction.

The rest of the paper is organized as follows. Section II details the GSP background and the problem formulation, and briefly describes preliminaries over EKF and KalmanNet algorithms. Section III derives GSP-EKF for scalable tracking in the graph frequency domain, assuming accurate knowledge of the SS model. Section IV presents GSP-KalmanNet, including its architecture and training method. Our numerical evaluation is provided in Section V, while Section VI concludes the paper.

Throughout the paper, we use boldface lower-case letters for vectors (e.g., $\mathbf{x}$), boldface upper-case letters for matrices (e.g., $\mathbf{X}$), and italic letters for sets (e.g., $\mathcal{V}$); the i th and (i, j) th entries of $\mathbf{x}$ and $\mathbf{X}$ are respectively denoted $[\mathbf{x}]_i$ and $[\mathbf{X}]_{i,j}$. The transpose, ℓ_2 norm, and gradient are denoted by $\{\cdot\}^\top$, $\|\cdot\|$, and $\nabla(\cdot)$, respectively, while $\text{diag}(\mathbf{x})$ is a diagonal matrix with $\mathbf{x}$ as its main diagonal. Finally, $\mathbf{1}$ is the all-ones vector, and $\mathbb{R}$ and $\mathbb{Z}^+$ are the sets of real and non-negative integer numbers, respectively.

II. SYSTEM MODEL AND PRELIMINARIES

In this section, we introduce the considered model of a dynamic system of graph signals, which sets the ground for our derivations of the tracking algorithms in the following sections. In particular, we present the GSP notations and graph frequency domain filtering in Subsection II-A. Then, we describe the SS formulation and the considered tracking of a networked data problem in Subsection II-B. Finally, we review the necessary preliminaries for our derivation of GSP-KalmanNet, which are the model-based EKF and data-aided KalmanNet of [53], in Subsections II-C and II-D, respectively.

A. GSP Background

We consider an undirected, connected, weighted graph $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathbf{W})$ representing, e.g., a power grid or a sensor network topology. Here, $\mathcal{V}$ and $\mathcal{E}$ are sets of nodes (vertices) and edges, respectively, and $\mathbf{W} \in \mathbb{R}^{N \times N}$ is the non-negative weighted adjacency matrix, where $N \triangleq |\mathcal{V}|$ is the number of nodes. If there is an edge $(i, j) \in \mathcal{E}$, the entry $[\mathbf{W}]_{i,j}$ represents the weight of the edge; otherwise, $[\mathbf{W}]_{i,j} = 0$. In general, graph signals are defined as follows [2]:

Definition II.1 (Graph signal): A graph signal $\mathbf{z}$ with entries in $\mathbb{R}$ is a function $\mathbf{z} : \mathcal{V} \rightarrow \mathbb{R}^N$ defined on the nodes of $\mathcal{G}$.

A common approach to designing filters for graph signals is based on the GFT. To formulate this, we use the Laplacian

matrix of the graph, which is defined as $\mathbf{L} \triangleq \text{diag}(\mathbf{W} \cdot \mathbf{1}) - \mathbf{W}$. The eigenvalue decomposition (EVD) of $\mathbf{L}$ is given by

$$\mathbf{L} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top. \quad (1)$$

In (1), $\mathbf{\Lambda}$ is a diagonal matrix consisting of the eigenvalues of $\mathbf{L}$, $\lambda_1 < \lambda_2 \leq \dots \leq \lambda_N$, the columns of $\mathbf{V}$ are comprised of the associated eigenvectors, and $\mathbf{V}^\top = \mathbf{V}^{-1}$. This EVD is used to define the GFT over $\mathcal{G}$ [3]:

Definition II.2 (GFT): The GFT of a graph signal defined over a graph $\mathcal{G}$ with Laplacian $\mathbf{L}$ is

$$\tilde{\mathbf{z}} \triangleq \mathbf{V}^\top \mathbf{z}. \quad (2)$$

Similarly, the inverse GFT is given by $\mathbf{V} \tilde{\mathbf{z}}$. Linear and shift-invariant graph filters of various graph shift operators play essential roles in GSP with various applications [60]. A common type of such a graph filter, which arises from the GFT in Definition II.2, is the *frequency domain graph filter*, defined as follows [61]:

Definition II.3: A frequency domain graph filter defined over $\mathcal{G}$ with Laplacian $\mathbf{L}$ is a function $g(\cdot)$ applied to the Laplacian, which allows the following EVD:

$$g(\mathbf{L}) = \mathbf{V} g(\mathbf{\Lambda}) \mathbf{V}^\top, \quad (3)$$

where $g(\mathbf{\Lambda})$ is a diagonal matrix. In addition, the diagonal elements of $g(\mathbf{\Lambda})$ that are associated with eigenvalues of $\mathbf{L}$ with a multiplicity greater than 1 are all equal, i.e., if $\lambda_{k+1} = \lambda_k$, then $g(\lambda_{k+1}) = g(\lambda_k)$.

One of the benefits of frequency domain graph filters stems from their reduced complexity when formulating their operation in the GFT domain. If $\tilde{\mathbf{y}}$ is the output of a frequency domain graph filter applied to a frequency domain graph signal $\tilde{\mathbf{x}}$, i.e., $\tilde{\mathbf{y}} = g(\mathbf{L}) \tilde{\mathbf{x}}$, then it holds that

$$[\tilde{\mathbf{y}}]_n = g(\lambda_n) [\tilde{\mathbf{x}}]_n, \quad \forall n \in 1, \dots, N, \quad (4)$$

where $g(\lambda_n)$ is the graph frequency response of the filter at graph frequency λ_n . Consequently, instead of filtering with a complexity that grows quadratically with N (multiplying by an $N \times N$ matrix in the vertex domain), it has a complexity that only grows linearly with N via (4).

B. Problem Formulation

We consider a dynamic system in discrete-time $t \in \mathbb{Z}^+$, with observations $\mathbf{y}_t \in \mathbb{R}^N$ and states $\mathbf{x}_t \in \mathbb{R}^N$ that are *graph signals* over the graph $\mathcal{G}$. The observations $\mathbf{y}_t$ represent a time sequence measured at each node, while $\mathbf{x}_t$ accommodates the latent state sequences at the nodes at time t . Accordingly, the n th entry of the processes $\mathbf{x}_t$ and $\mathbf{y}_t$ creates a time series evolving on node n of the graph [28]. While we consider graph signals with scalar values per node, the formulation readily extends to multivariate values at each node.

1) SS Model: The dynamics are characterized by a nonlinear, continuous SS model in discrete-time:

$$\mathbf{x}_t = \mathbf{f}_t(\mathbf{L}, \mathbf{x}_{t-1}) + \mathbf{e}_t, \quad (5a)$$

$$\mathbf{y}_t = \mathbf{h}_t(\mathbf{L}, \mathbf{x}_t) + \mathbf{v}_t. \quad (5b)$$

In (5), $\mathbf{e}_t \in \mathbb{R}^N$ and $\mathbf{v}_t \in \mathbb{R}^N$ are temporally i.i.d., zero-mean, and mutually independent state-evolution and measurement noises, respectively, at time t . In addition, $\mathbf{Q}$ and $\mathbf{R}$ are the covariance matrices of $\mathbf{e}_t$ and $\mathbf{v}_t$, respectively. The nonlinear state evolution $\mathbf{f}_t: \mathbb{R}^{N \times N} \times \mathbb{R}^N \mapsto \mathbb{R}^N$ and measurement function $\mathbf{h}_t: \mathbb{R}^{N \times N} \times \mathbb{R}^N \mapsto \mathbb{R}^N$ generally depend on the graph via the Laplacian matrix, $\mathbf{L}$.

This model comprises a broad family of statistical tracking problems over graphs. For example, a core problem of power system analysis is the recovery of voltages from power measurements, where both the voltages and the powers can be considered as graph signals [17], [62], [63] and the Laplacian matrix is the susceptance matrix. This tracking problem is one of the settings considered in Section V.

2) *Filtering Problem*: Our goal is to develop a filtering algorithm for tracking the latent graph signal $\mathbf{x}_t$ from the observed graph signals $\{\mathbf{y}_\tau\}_{\tau \leq t}$ for each time instance t . The performance of a given estimator, denoted $\hat{\mathbf{x}}_t$, is measured using the average MSE. That is, for a time sequence of length T , the MSE is computed as

$$\text{MSE} = \frac{1}{T} \sum_{t=1}^T \mathbb{E}\{\|\hat{\mathbf{x}}_t - \mathbf{x}_t\|^2\}. \quad (6)$$

We focus on tracking with *partially known SS models*. Specifically, we aim to cope with the following challenges:

- C.1 The size of the graph signal, N , which is the number of nodes, can be very large, causing high complexity and affecting real-time applicability.
- C.2 The graph topology, which can be captured by the Laplacian matrix $\mathbf{L}$, may be misspecified, both in its connectivity pattern and the weights.
- C.3 The distributions of the noise signals, $\mathbf{e}_t$ and $\mathbf{v}_t$, are unknown and may be non-Gaussian.
- C.4 The state-evolution function, $\mathbf{f}(\cdot)$, and the measurement function, $\mathbf{h}(\cdot)$, are available, but may be mismatched.

Challenge C.1 is commonly encountered in graph signals due to the growing size of typical virtual and physical networks. Challenge C.2 corresponds to the case where one is unaware of the true weights of some of the edges, possibly even wrongly assuming the presence or absence of some edges in a graph. Challenge C.3 stems from the fact that stochasticity in graphical data is often non-Gaussian, while the presence of mismatches in the model functions in Challenge C.4 is common in real-world tracking, where the physical phenomena cannot be fully described by simple mathematical equations.

To cope with the various unknown characteristics of (5) encapsulated in Challenges C.2–C.4, we utilize a labeled dataset. The data-set is comprised of D trajectories, in which the d th trajectory is a sequence of paired graph signals of observations and states in length T_d . The data-set is given by:

$$\mathcal{D} \triangleq \left\{ \left\{ \mathbf{x}_t^{(d)}, \mathbf{y}_t^{(d)} \right\}_{t=1}^{T_d} \right\}_{d=1}^D, \quad (7)$$

where $\mathbf{x}_t^{(d)}$ and $\mathbf{y}_t^{(d)}$ are the state and observation vectors, respectively, at time t and of trajectory d .

Our proposed algorithm for tackling Challenges C.1–C.4 is detailed in Section IV. The design follows a model-based DL methodology [50], [51], [64], where DL tools are used to augment and empower existing model-based algorithms though replacing the parts that rely on missing domain knowledge with learnable components. Our method builds upon the properties of GSP, specifically in the graph frequency domain, which enables to reduce the classical EKF complexity when dealing with large signals (Challenge C.1). To describe the proposed algorithm, in the following two subsections, we review preliminaries of tracking in partially known SS models in a data-driven fashion, building upon the recently proposed KalmanNet of [53] (presented in Subsection II-D), which augments the classic EKF (presented in Subsection II-C).

C. Preliminaries: EKF

Tracking signals in SS models in regular domains (i.e., not over irregular graphs) is widely studied (see, e.g., [10, Ch. 10]). One of the most common algorithms, which is suitable for nonlinear models where the noise signals are Gaussian, the SS model is fully known, and the dimensions of the system are relatively small, i.e., in the absence of Challenges C.1–C.4, is the EKF [65]. The algorithm follows the operation of the KF, combining prediction based on the previous estimate with an update based on the current observation, while extending it to nonlinear SS models. In particular, the EKF first predicts the next state and observation based on $\hat{\mathbf{x}}_{t-1}$ via

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{f}(\hat{\mathbf{x}}_{t-1}); \quad \hat{\mathbf{y}}_{t|t-1} = \mathbf{h}(\hat{\mathbf{x}}_{t|t-1}). \quad (8)$$

Then, the initial prediction is updated using a matrix $\mathcal{K}_t$, known as the Kalman gain (KG), which dictates the balance between relying on the state evolution function $\mathbf{f}(\cdot)$ through (8) and the current observation $\mathbf{y}_t$. In particular, the estimator is computed as

$$\hat{\mathbf{x}}_t = \mathcal{K}_t \cdot \Delta \mathbf{y}_t + \hat{\mathbf{x}}_{t|t-1}, \quad (9)$$

where $\Delta \mathbf{y}_t \triangleq \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}$. The KG is calculated via

$$\mathcal{K}_t = \hat{\Sigma}_{t|t-1} \cdot \hat{\mathbf{H}}_t^\top \cdot \hat{\mathbf{S}}_{t|t-1}^{-1}, \quad (10)$$

where $\hat{\Sigma}_{t|t-1}$ and $\hat{\mathbf{S}}_{t|t-1}$ are the covariance matrices of the state prediction $\hat{\mathbf{x}}_{t|t-1}$ and observation prediction $\hat{\mathbf{y}}_{t|t-1}$, respectively. These matrices are calculated via

$$\hat{\Sigma}_{t|t-1} = \hat{\mathbf{F}}_t \cdot \hat{\Sigma}_{t-1} \cdot \hat{\mathbf{F}}_t^\top + \mathbf{Q}, \quad (11a)$$

$$\hat{\mathbf{S}}_{t|t-1} = \hat{\mathbf{H}}_t \cdot \hat{\Sigma}_{t|t-1} \cdot \hat{\mathbf{H}}_t^\top + \mathbf{R}, \quad (11b)$$

where it is assumed that $\hat{\mathbf{S}}_{t|t-1}$ is a non-singular matrix. The matrices $\hat{\mathbf{F}}_t$ and $\hat{\mathbf{H}}_t$ are the linearized approximations of $\mathbf{f}(\cdot)$ and $\mathbf{h}(\cdot)$, respectively, obtained using their Jacobian matrices evaluated at $\hat{\mathbf{x}}_{t-1}$ and $\hat{\mathbf{x}}_{t|t-1}$ (see [10, Ch. 10]), i.e.,

$$\hat{\mathbf{F}}_t = \nabla_{\mathbf{x}} \mathbf{f}(\hat{\mathbf{x}}_{t-1}); \quad \hat{\mathbf{H}}_t = \nabla_{\mathbf{x}} \mathbf{h}(\hat{\mathbf{x}}_{t|t-1}). \quad (12)$$

Finally, the error covariance is updated as follows:

$$\hat{\Sigma}_t = (\mathbf{I} - \mathcal{K}_t \cdot \hat{\mathbf{H}}_t) \hat{\Sigma}_{t|t-1} (\mathbf{I} - \mathcal{K}_t \cdot \hat{\mathbf{H}}_t)^\top + \mathcal{K}_t \cdot \mathbf{R} \cdot \mathcal{K}_t^\top. \quad (13)$$

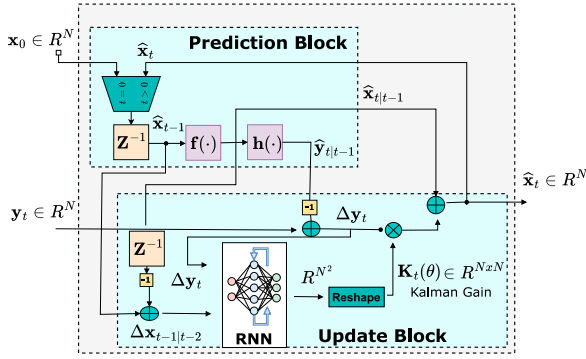


Fig. 1. KalmanNet block diagram.

D. Preliminaries: KalmanNet

Challenges C.1-C.4 significantly limit the applicability of the EKF for the setup detailed in Subsection II-B. For instance, the distributions of the noise signals must be known in order to compute the matrices in (11). KalmanNet, proposed in [53], is a hybrid model-based/data-driven architecture, which leverages data as in (7) to tackle Challenges C.3-C.4 (but not Challenges C.1-C.2). KalmanNet builds on the insight that the missing and mismatched domain knowledge is encapsulated in the computation of the KG, $\mathcal{K}_t$ in (10). Consequently, it augments the EKF with a DL component by replacing the computation of the KG with a dedicated RNN-based architecture. The KG is computed using the features that are selected as ones that are indicative of the state and measurement noise statistics. For example, these features can be chosen as $\Delta \mathbf{y}_t \triangleq \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}$ and $\Delta \mathbf{x}_t \triangleq \hat{\mathbf{x}}_t - \hat{\mathbf{x}}_{t-1}$. Accordingly, KalmanNet sets the KG estimation to be:

$$\mathcal{K}_t(\theta) = \mathbf{g}_\theta^{\text{KG}}(\Delta \mathbf{y}_t, \Delta \mathbf{x}_t), \quad (14)$$

where $\mathbf{g}_\theta^{\text{KG}}(\cdot)$ denotes the RNN mapping with parameters θ . The rest of the filtering operation is the same via (8), while replacing the state estimate in (9) with

$$\hat{\mathbf{x}}_t = \mathcal{K}_t(\theta) \cdot \Delta \mathbf{y}_t + \hat{\mathbf{x}}_{t|t-1}. \quad (15)$$

The weights of the RNN, θ , were learned from a data-set (7), based on the regularized MSE loss function (6). During training, the gradients of the loss with respect to θ propagate through the entire EKF pipeline to learn the computation of the KG as the one yielding the state estimate best matching the data in the sense of the MSE loss. By doing so, KalmanNet converts the EKF into a trainable discriminative model [59], where the data $\mathcal{D}$ is used to directly learn the KG, bypassing the need to enforce any model over the noise statistics and able to handle domain knowledge mismatches as described in Challenges C.3-C.4. An illustration of the algorithm block diagram can be seen in Fig. 1.

KalmanNet was shown to successfully cope with Challenges C.3-C.4 when applied to signals with relatively small dimensionality [53]. However, in the considered setup, where the cardinality of the graph signals (i.e., N) can be very large, KalmanNet may become highly complex (as its number of

RNN neurons grows with N^3 [53]), significantly impacting the computational burden and data requirements of its training and inference. Since the state is a high-dimensional graph signal, one cannot mitigate complexity by mapping the observations into a low-dimensional subspace, as done in [58], [66]. This indicates that in order to jointly tackle Challenges C.1-C.4, one must account for the graph structure of the signals in the algorithm design, which is the basic principle in our derivation of GSP-KalmanNet in the following sections.

III. TRACKING IN THE GRAPH FREQUENCY DOMAIN

In this section, we design our tracking algorithm for the problem formulated in Subsection II-B in two stages. We first focus only on Challenge C.1, i.e., we assume that the SS model is fully known, and thus data is not exploited; then, we extend this approach by using data and DL tools to also cope with Challenges C.2-C.4 in Section IV. We first present the EKF in the graph frequency domain in Subsection III-A. Then, we use this representation to formulate GSP-EKF in Subsection III-B, which reduces the computational complexity. An analysis of the complexity of GSP-EKF and conditions for coincidence with the EKF are given in Subsection III-C.

A. EKF in the Graph Frequency Domain

The EKF for the SS model in (5) (assuming known noise distributions) can be implemented in the graph frequency domain. To this end, we transform (5) to the graph frequency domain by left-multiplying these equations by $\mathbf{V}^\top$. According to (2), we write the GFTs of the different graph signals as $\tilde{\mathbf{x}}_t = \mathbf{V}^\top \mathbf{x}_t$, $\tilde{\mathbf{e}}_t = \mathbf{V}^\top \mathbf{e}_t$, $\tilde{\mathbf{y}}_t = \mathbf{V}^\top \mathbf{y}_t$, and $\tilde{\mathbf{v}}_t = \mathbf{V}^\top \mathbf{v}_t$. With a slight abuse of notation, we denote

$$\tilde{\mathbf{f}}_t(\mathbf{L}, \mathbf{V}\tilde{\mathbf{x}}_{t-1}) \triangleq \mathbf{V}^\top \mathbf{f}_t(\mathbf{L}, \mathbf{x}_{t-1}), \quad (16a)$$

$$\tilde{\mathbf{h}}_t(\mathbf{L}, \mathbf{V}\tilde{\mathbf{x}}_t) \triangleq \mathbf{V}^\top \mathbf{h}_t(\mathbf{L}, \mathbf{x}_t). \quad (16b)$$

Since $\mathbf{V}^\top \mathbf{V} = \mathbf{I}$, we obtain the graph frequency SS model

$$\tilde{\mathbf{x}}_t = \tilde{\mathbf{f}}_t(\mathbf{L}, \mathbf{V}\tilde{\mathbf{x}}_{t-1}) + \tilde{\mathbf{e}}_t, \quad (17a)$$

$$\tilde{\mathbf{y}}_t = \tilde{\mathbf{h}}_t(\mathbf{L}, \mathbf{V}\tilde{\mathbf{x}}_t) + \tilde{\mathbf{v}}_t. \quad (17b)$$

Using the model assumptions given after (5), it can be verified that the state-evolution and measurement noises in the graph frequency domain, $\tilde{\mathbf{e}}_t \in \mathbb{R}^N$ and $\tilde{\mathbf{v}}_t \in \mathbb{R}^N$, respectively, are mutually independent at time t , and each one of them also has independency between different time samples, since they are obtained from the noises in (5) by a linear transformation. In addition, they still have zero mean and known covariance matrices that are now given by

$$\tilde{\mathbf{R}} \triangleq \mathbb{E}\{\tilde{\mathbf{v}}_t \tilde{\mathbf{v}}_t^\top\} = \mathbb{E}\{\mathbf{V}^\top \mathbf{v}_t (\mathbf{V}^\top \mathbf{v}_t)^\top\} = \mathbf{V}^\top \mathbf{R} \mathbf{V} \quad (18)$$

and

$$\tilde{\mathbf{Q}} \triangleq \mathbb{E}\{\tilde{\mathbf{e}}_t \tilde{\mathbf{e}}_t^\top\} = \mathbb{E}\{\mathbf{V}^\top \mathbf{e}_t (\mathbf{V}^\top \mathbf{e}_t)^\top\} = \mathbf{V}^\top \mathbf{Q} \mathbf{V}, \quad (19)$$

where we used the fact that $\mathbf{V}$ is a deterministic matrix, as well as the definitions of $\mathbf{Q}$ and $\mathbf{R}$ from Subsection II-B1.

The EKF in the graph frequency domain can be developed directly from the model in (17), in a similar manner to the EKF in Subsection II-C. The resulting formulation of the KG and the error covariance are stated in the following lemma:

Lemma 1: The KG and the error covariance of the EKF in the graph frequency domain are given by

$$\tilde{\mathcal{K}}_t = \tilde{\Sigma}_{t|t-1} \cdot \tilde{\mathbf{H}}_t^\top \cdot \tilde{\mathbf{S}}_{t|t-1}^{-1}, \quad (20)$$

and

$$\tilde{\Sigma}_{t|t} = (\mathbf{I} - \tilde{\mathcal{K}}_t \cdot \tilde{\mathbf{H}}_t) \tilde{\Sigma}_{t|t-1} (\mathbf{I} - \tilde{\mathcal{K}}_t \cdot \tilde{\mathbf{H}}_t)^\top + \tilde{\mathcal{K}}_t \cdot \tilde{\mathbf{R}} \cdot \tilde{\mathcal{K}}_t^\top, \quad (21)$$

where

$$\tilde{\Sigma}_{t|t-1} = \mathbf{V}^\top \hat{\Sigma}_{t|t-1} \mathbf{V}, \quad (22)$$

$$\tilde{\mathbf{S}}_{t|t-1} = \mathbf{V}^\top \hat{\mathbf{S}}_{t|t-1} \mathbf{V}, \quad (23)$$

$\tilde{\mathbf{H}}_t = \mathbf{V}^\top \hat{\mathbf{H}}_t \mathbf{V}$ is the Jacobian matrix, $\hat{\mathbf{H}}_t$ is defined in (12).

Proof: First, we note that the graph frequency SS model from (17)-(17b) is obtained from the SS model in (5) by left-multiplying the equations (5) by $\mathbf{V}^\top$. Since this transformation to the graph frequency domain in Definition II.2 is an affine unitary transformation, the EKF in the graph frequency domain can be obtained from the standard (vertex-domain) EKF in Subsection II-C by left-multiplying (8)-(13) by $\mathbf{V}^\top$, right-multiplying (10)-(13) by $\mathbf{V}$, and using Definition II.2. In particular, by left- and right-multiplying (10) by $\mathbf{V}^\top$ and $\mathbf{V}$, respectively, and by using the fact that $\mathbf{V}\mathbf{V}^\top = \mathbf{I}$, we obtain

$$\mathbf{V}^\top \mathcal{K}_t \mathbf{V} = \mathbf{V}^\top \hat{\Sigma}_{t|t-1} \cdot \mathbf{V} \mathbf{V}^\top \cdot \hat{\mathbf{H}}_t^\top \cdot \mathbf{V} \mathbf{V}^\top \cdot \hat{\mathbf{S}}_{t|t-1}^{-1} \mathbf{V}. \quad (24)$$

Thus, we can define the Kalman gain in the graph frequency domain as in (20). In a similar manner, (13), together with the graph frequency domain error covariance matrices from (18)-(19), imply that the error covariance in the graph frequency domain is given by (21), thus proving the lemma. ■

The graph frequency domain EKF above is the classic EKF formulated for the SS model in (17). Thus, it is still subject to the complexity challenge C.1: computing the KG in (20) involves inverting an $N \times N$ matrix, and its application involves multiplying the observations and states by an $N \times N$ matrix, $\mathbf{V}^\top$, when transforming to the graph frequency domain. These operations can be computationally costly for large networks, which motivates the derivation of GSP-EKF.

B. GSP-EKF

The GSP-EKF algorithm is based on working in the graph frequency domain and designing the KG as a graph filter. In the following, we develop the KG of the GSP-EKF algorithm by finding the graph filter that minimizes the computed covariance matrix of the estimator $\hat{\mathbf{x}}_t$.

The GSP-EKF algorithm designs the KG as a graph filter. Thus, we restrict the KG to take the form in Definition II.3:

$$\mathcal{K}_t = \mathbf{V} g(\Lambda) \mathbf{V}^\top, \quad (25)$$

where $\mathbf{V}$ and Λ are obtained from the EVD of $\mathbf{L}$ in (1), and $g(\cdot)$ is a graph filter, as defined in (3). By (25), this is equivalent to

using the EKF in the graph frequency domain, while replacing $\tilde{\mathcal{K}}_t$ from (20) by a diagonal gain:

$$\tilde{\mathcal{K}}_t = g(\Lambda). \quad (26)$$

Substituting (26) in (21) leads to the following graph frequency domain covariance matrix under graph-filter gain:

$$\tilde{\Sigma}_{t|t}^{\text{GSP}} = (\mathbf{I} - g(\Lambda) \cdot \tilde{\mathbf{H}}_t) \tilde{\Sigma}_{t|t-1} (\mathbf{I} - g(\Lambda) \cdot \tilde{\mathbf{H}}_t)^\top + g(\Lambda) \cdot \tilde{\mathbf{R}} \cdot g(\Lambda). \quad (27)$$

Restricting the KG to be a graph filter implies that it generally cannot be computed as in the standard EKF, i.e., via (20). The GSP-EKF thus sets the filter to minimize the computed covariance matrix of the estimator $\hat{\mathbf{x}}_t$ over the subset of GSP KGs in the form of (26). That is, the KG in the graph frequency domain is set to minimize the trace of (27), i.e.,

$$\tilde{\mathcal{K}}_t^{\text{GSP}} = \arg \min_{g(\Lambda) \in \mathcal{G}_N} \left\{ \text{trace}(\tilde{\Sigma}_{t|t}^{\text{GSP}}) \right\}, \quad (28)$$

where $\mathcal{G}_N$ is the set of diagonal matrices of size $N \times N$.

By substituting (27) in (28) using the trace operator properties, the minimization from (28) can be written as

$$\tilde{\mathcal{K}}_t^{\text{GSP}} = \arg \min_{g(\Lambda) \in \mathcal{G}_N} \left\{ \text{trace}((\mathbf{I} - g(\Lambda) \cdot \tilde{\mathbf{H}}_t) \tilde{\Sigma}_{t|t-1} \cdot (\mathbf{I} - g(\Lambda) \cdot \tilde{\mathbf{H}}_t)^\top) + \text{trace}(g(\Lambda) \cdot \tilde{\mathbf{R}} \cdot g(\Lambda)) \right\}. \quad (29)$$

By equating the derivative of the r.h.s. of (29) with respect to $g(\Lambda) \in \mathcal{G}_N$ to zero, we obtain that the solution of (29) is

$$\tilde{\mathcal{K}}_t^{\text{GSP}} = \text{ddiag}(\tilde{\Sigma}_{t|t-1} \cdot \tilde{\mathbf{H}}_t^\top) \cdot (\text{ddiag}(\tilde{\mathbf{H}}_t \cdot \tilde{\Sigma}_{t|t-1} \cdot \tilde{\mathbf{H}}_t^\top + \tilde{\mathbf{R}}))^{-1}, \quad (30)$$

where $\text{ddiag}(\mathbf{D})$ is a diagonal matrix with the same diagonal as $\mathbf{D}$. The GSP-EKF is summarized in Algorithm 1. Note that implementing Algorithm 1 while replacing $\tilde{\mathcal{K}}_t^{\text{GSP}}$ with the KG $\mathcal{K}_t$ in (20) results in the EKF in the graph frequency domain from Subsection III-A.

C. GSP-EKF Discussion

In this subsection, we discuss the properties of the GSP-EKF in Algorithm 1, including its computational complexity. These properties will be used to develop and analyze the properties of the GSP-KalmanNet, as detailed in Subsection IV-D.

1) *Motivation:* An advantage of implementing the EKF in the graph frequency domain, (via the algorithm from Subsection III-A or from Subsection III-B) compared with the EKF appears for cases where $\mathbf{x}_t$ and $\mathbf{e}_t$ are graph bandlimited signals [2] with a cutoff graph frequency N_s . In these cases, we can use $[\hat{\mathbf{x}}_{t|s}]_n = 0, \forall n > N_s, t, s \in \mathbb{Z}$ and track only the first N_s elements of $\hat{\mathbf{x}}_t$ at each time step. Moreover, the GSP-EKF enforces the KG to be a graph filter, which effectively introduces regularization that leverages the underlying graphical information. This graph filter can be further modified by using parametric graph filter design techniques [67], which enables the development of a distributed, local implementation [68]. The integration of graph information via graph filters in the

Algorithm 1: GSP-EKF at time t

Input: Laplacian $\mathbf{L}$; functions $\mathbf{f}_t(\cdot)$, $\mathbf{h}_t(\cdot)$;
SS model parameters $\mathbf{Q}$ and $\mathbf{R}$;

- 1 Compute EVD: $\mathbf{L} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$;
- 2 Compute $\tilde{\mathbf{Q}} = \mathbf{V}^\top \mathbf{Q} \mathbf{V}$ and $\tilde{\mathbf{R}} = \mathbf{V}^\top \mathbf{R} \mathbf{V}$;
- 3 Compute linearized $\tilde{\mathbf{F}}_t$ and $\tilde{\mathbf{H}}_t$ via (12);
- 4 Compute $\tilde{\mathbf{F}}_t = \mathbf{V}^\top \tilde{\mathbf{F}}_t \mathbf{V}$ and $\tilde{\mathbf{H}}_t = \mathbf{V}^\top \tilde{\mathbf{H}}_t \mathbf{V}$;
- 5 Prediction step:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{V}^\top \mathbf{f}_t(\mathbf{L}, \mathbf{V} \hat{\mathbf{x}}_{t-1}) \quad (31)$$

$$\hat{\mathbf{y}}_{t|t-1} = \mathbf{V}^\top \mathbf{h}_t(\mathbf{L}, \mathbf{V} \hat{\mathbf{x}}_{t|t-1}) \quad (32)$$

$$\tilde{\Sigma}_{t|t-1} = \tilde{\mathbf{F}}_t \tilde{\Sigma}_{t-1} \tilde{\mathbf{F}}_t^\top + \tilde{\mathbf{Q}} \quad (33)$$

- 6 Update step:

- 7 Compute the KG $\tilde{\mathcal{K}}_t^{(\text{GSP})}$ from (30);

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t-1} + \tilde{\mathcal{K}}_t^{(\text{GSP})} (\hat{\mathbf{y}}_t - \hat{\mathbf{y}}_{t|t-1}) \quad (34)$$

$$\begin{aligned} \tilde{\Sigma}_t &= (\mathbf{I} - \tilde{\mathcal{K}}_t^{(\text{GSP})} \cdot \tilde{\mathbf{H}}_t) \tilde{\Sigma}_{t|t-1} (\mathbf{I} - \tilde{\mathcal{K}}_t^{(\text{GSP})} \cdot \tilde{\mathbf{H}}_t)^\top \\ &\quad + \tilde{\mathcal{K}}_t^{(\text{GSP})} \cdot \tilde{\mathbf{R}} \cdot \tilde{\mathcal{K}}_t^{(\text{GSP})} \end{aligned} \quad (35)$$

Return $\hat{\mathbf{x}}_t = \mathbf{V} \hat{\mathbf{x}}_t$

GSP-EKF has the potential to improve the performance in terms of MSE, interpretability, robustness, complexity, flexibility, and stability compared to the conventional EKF. These advantages have previously been substantiated for static estimation that is based on GSP methodologies [18], [20].

2) *Computational Complexity:* The GSP-EKF is particularly designed to cope with the excessive complexity of applying the EKF to high-dimensional graph signals (Challenge C.1). We thus commence our analysis of the GSP-EKF with an analysis of the computational savings, as compared with the standard EKF. Unlike the standard EKF, the GSP-EKF does not require non-diagonal matrix inversion at each time step for computing the KG, as in (10). In fact, the KG of the GSP-EKF can be calculated in a vectorized form with a complexity of $\mathcal{O}(N)$ multiplications instead of $\mathcal{O}(N^3)$ (due to inverting an $N \times N$ matrix) for the conventional EKF. It is noted that the transformation of the system to the graph frequency domain requires the computation of the EVD of the Laplacian matrix of order $\mathcal{O}(N^3)$ (Step 1 of Algorithm 1). However, this calculation can be carried out offline, as it only has to be done once for a given graph, and possibly with low-complexity methods [69]. Thus, its complexity is considered separately from that of the online processing of each incoming sample, which is the core source of processing latency and thus the focus of our complexity analysis. Finally, it should be noted that the transformation of each signal to the graph frequency domain requires multiplication by $N \times N$ matrix, $\mathbf{V}^\top$ ($\mathcal{O}(N^2)$), as opposed to $\mathcal{O}(N^3)$ in the unconstrained EKF.

3) *Orthogonal Graph Frequencies:* While the GSP-EKF is in general different from the EKF, in some cases, the filters coincide. One such setting of *orthogonal graph frequencies* is characterized by the following theorem.

Theorem 1: The GSP-EKF coincides with the EKF if

- 1) The matrices, $\mathbf{Q}$ and $\mathbf{R}$, are diagonalizable by $\mathbf{V}$. Thus, $\tilde{\mathbf{Q}}$, and $\tilde{\mathbf{R}}$ are diagonal matrices.
- 2) The functions $\mathbf{f}_t(\mathbf{L}, \mathbf{x})$ and $\mathbf{h}_t(\mathbf{L}, \mathbf{x})$ are separable in the graph frequency domain, i.e., for all $n = 1, \dots, N$

$$[\tilde{\mathbf{f}}_t(\mathbf{L}, \mathbf{V} \tilde{\mathbf{x}})]_n = [\tilde{\mathbf{f}}_t(\mathbf{L}, [\tilde{\mathbf{x}}]_n \mathbf{V}_{:,n})]_n, \quad (36a)$$

$$[\tilde{\mathbf{h}}_t(\mathbf{L}, \mathbf{V} \tilde{\mathbf{x}})]_n = [\tilde{\mathbf{h}}_t(\mathbf{L}, [\tilde{\mathbf{x}}]_n \mathbf{V}_{:,n})]_n, \quad (36b)$$

where $\mathbf{V}_{:,n}$ is the n th column of $\mathbf{V}$, and $[\tilde{\mathbf{x}}]_n$ is the n th element of $\tilde{\mathbf{x}}$

Proof: The proof is given in the appendix. ■

Under the conditions of Theorem 1, the KG of the conventional EKF is a graph filter, and the information for estimating the different elements of $\tilde{\mathbf{x}}_t$ is decoupled. Condition 1 implies that the elements of the noise signals are uncorrelated in the graph frequency domain for any t . Condition 2 requires the state evolution and measurement functions to be separable in the graph frequency domain. This situation occurs, e.g., when the states and measurements are linear outputs of graph filters [18]. In addition, the fact that the GSP-EKF coincides with the EKF implies that for the special case of Gaussian linear SS models, Algorithm 1 is reduced to the MSE-optimal Kalman filter, which is equivalent in this case to the graph frequency domain KF proposed in [28].

IV. GSP-KALMANNET

The GSP-EKF derived above only partially copes with the challenges discussed in Subsection II-B. Specifically, it only tackles Challenge C.1 (Complexity). However, its suitability for tracking graph signals indicates that in order to jointly tackle Challenges C.1-C.4, we should leverage data (7) and convert the GSP-EKF into a hybrid discriminative model-based/data-driven architecture [59], in the same spirit of converting the EKF to the KalmanNet in [53]. The resulting GSP-KalmanNet algorithm presented in this section is a hybrid, interpretable, and data-efficient scheme for tracking graph signals in partially known nonlinear dynamics. To introduce GSP-KalmanNet, we begin by explaining its high level operation in Subsection IV-A, and detail the features processed by its RNN and its specific architecture in Subsection IV-B. Then, we present how GSP-KalmanNet is trained in Subsection IV-C, and provide a discussion in Subsection IV-D.

A. High Level Architecture

While the design of GSP-KalmanNet follows the methodology used in formulating KalmanNet (see Subsection II-D), its architecture differs from that proposed in [53]. This stems from the fact that the complexity of KalmanNet, which is based on the conventional EKF, scales cubically with the dimensions of the signals ($\mathcal{O}(N^3)$), and thus, cannot be used to track graph signals in large networks. Previous adaptations of KalmanNet for high-dimensional signals, e.g., [58], were tailored for low-dimensional states and only high-dimensional observations. Accordingly, we derive here the GSP-KalmanNet from the GSP-EKF detailed in Subsection III-B, leveraging its

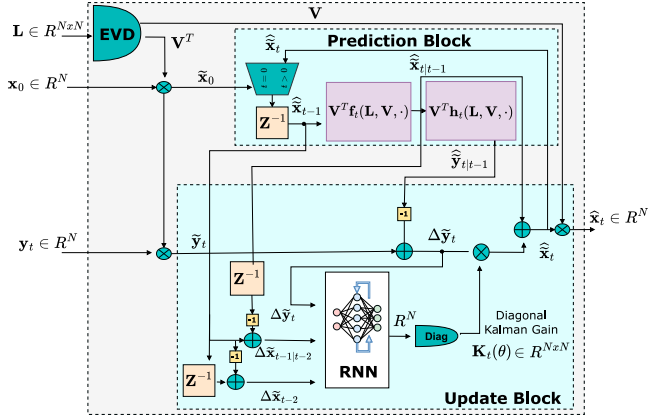


Fig. 2. GSP-KalmanNet block diagram.

suitability to track graph signals with affordable complexity, while augmenting its specific computations that are based on unavailable knowledge with trainable components.

As detailed in Subsection III-A, the functions $\tilde{f}_t(\cdot)$ and $\tilde{h}_t(\cdot)$ are known (albeit possibly inaccurately); yet the distribution of the noise terms is unavailable, and potentially non-Gaussian. Their missing statistical moments are used in GSP-EKF only to compute the KG. Thus, we design GSP-KalmanNet to learn the KG from data by combining the learned KG with the overall GSP-EKF flow. In particular, we use a trainable RNN to output the coefficient of a frequency domain graph filter, enforcing a diagonal matrix manipulation in the graph frequency domain. This architecture is illustrated in Fig. 2.

For a trained architecture, i.e., when the parameters of the RNN denoted θ are set, GSP-KalmanNet operates in a similar manner to the GSP-EKF in Algorithm 1. Specifically, at each time instance t , GSP-KalmanNet estimates $\hat{\mathbf{x}}_t$ in two steps:

- 1) In the *prediction* step, a prior estimator for the current state and observation $\hat{\mathbf{x}}_{t|t-1}$, and $\hat{\mathbf{y}}_{t|t-1}$ are computed from the previous posterior $\hat{\mathbf{x}}_{t-1}$, in the graph frequency domain by (37) and (38), respectively.
- 2) In the *update* step, the computation of the graph filter comprising the diagonal KG is not given explicitly. Rather, it is learned from data using the internal RNN, that is thus denoted $\tilde{\mathcal{K}}_t(\theta)$, then update the estimate by using the given observation $\tilde{\mathbf{y}}_t$ in (39). Unlike GSP-EKF, the inherent memory of RNNs makes it possible to implicitly track the second-order statistical moments, bypassing the need to know the noise distribution.

Since the state is tracked in the graph frequency domain, the vertex domain representation is obtained by the inverse GFT. This inference procedure of GSP-KalmanNet is summarized in Algorithm 2, and is illustrated in the block diagram in Fig. 2.

B. Neural Network Architecture

The trainable architecture of the internal module at step 4 of Algorithm 2, produces an $N \times 1$ vector comprising the diagonal of the graph frequency domain KG denoted $\tilde{\mathcal{K}}_t(\theta)$. We use an RNN for the training approach, as the recursive nature of the KG computation indicates that its learned module should

Algorithm 2: GSP-KalmanNet inference at time t

Init: Trained RNN architecture;

Input: Laplacian $\mathbf{L}$; functions $\tilde{f}_t(\cdot)$, $\tilde{h}_t(\cdot)$;

1 Compute EVD: $\mathbf{L} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$;

2 Prediction step:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{V}^\top \tilde{f}_t(\mathbf{L}, \mathbf{V} \hat{\mathbf{x}}_{t-1}) \quad (37)$$

$$\hat{\mathbf{y}}_{t|t-1} = \mathbf{V}^\top \tilde{h}_t(\mathbf{L}, \mathbf{V} \hat{\mathbf{x}}_{t|t-1}) \quad (38)$$

3 Update step:

4 Apply the RNN to compute KG $\tilde{\mathcal{K}}_t(\theta)$ by (14);

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t-1} + \tilde{\mathcal{K}}_t(\theta) \cdot (\tilde{\mathbf{y}}_t - \hat{\mathbf{y}}_{t|t-1}) \quad (39)$$

Return $\hat{\mathbf{x}}_t = \mathbf{V} \hat{\mathbf{x}}_t$

TABLE I
RNN ARCHITECTURE

Layer	Output Size
Input	$3N$
FC + ReLU	$24N$
GRU	$20N$
GRU	$20N$
FC + ReLU	$4N$
FC	N

involve an internal memory element. We specifically employ an architecture comprised of a simple gated recurrent unit (GRU) [70] with input shaping and output shaping fully connected (FC) layers. The size of the GRU hidden state is determined by a factor of $2N$, where $2N$ is the joint dimensionality of the tracked moments $\hat{\mathbf{S}}_{t|t-1}$ and $\hat{\mathbf{S}}_{t|t-1}$ in (11a) and (11b), respectively. While the architecture used in Section V and presented in Table I uses 2 GRU layers, one can utilize multiple layers to increase capacity and abstractness.

Since the computation of the KG should be based on features that are indicative of the noise statistics in the SS, we use the following signals as the input to the RNN:

F1 *Innovation process* $\Delta \tilde{\mathbf{y}}_t = \tilde{\mathbf{y}}_t - \hat{\mathbf{y}}_{t|t-1}$.

F2 *Forward evolution difference* $\Delta \tilde{\mathbf{x}}_{t-2} = \hat{\mathbf{x}}_{t-1} - \hat{\mathbf{x}}_{t-2}$. This quantity represents the difference between two consecutive posterior state estimates.

F3 *Forward update difference* $\Delta \tilde{\mathbf{x}}_{t-1|t-2} = \hat{\mathbf{x}}_{t-1} - \hat{\mathbf{x}}_{t-1|t-2}$. This is the difference between the previous posterior state estimator and the prior state estimator.

All these features are in the graph frequency domain, where Feature F1 is informative about the measurement noise, while Features F2 and F3 indicate the uncertainty of the state estimator. Accordingly, the overall input is a $3N \times 1$ vector, as presented in Table I.

C. Training Algorithm

The trainable component of GSP-KalmanNet is the RNN outputting the KG; the remaining computations follow the GSP-EKF as stated in Algorithm 2. Nonetheless, instead of training the RNN based on the gap between its output and

Algorithm 3: Training GSP-KalmanNet

Init: Fix learning rate $\mu > 0$; epochs $i_{\max}$;
Initial guess θ ;
Input: Laplacian $\mathbf{L}$; functions $\mathbf{f}_t(\cdot)$, $\mathbf{h}_t(\cdot)$;
training set $\mathcal{D}$ (7);

- 1 Compute EVD: $\mathbf{L} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$;
- 2 **for** $i = 0, 1, \dots, i_{\max} - 1$ **do**
- 3 Randomly divide $\mathcal{D}$ into Q batches $\{\mathcal{D}_q\}_{q=1}^Q$;
- 4 **for** $q = 1, \dots, Q$ **do**
- 5 Apply Algorithm 2 to estimate $\hat{\mathbf{x}}_t$;
- 6 Compute batch loss $\mathcal{L}_{\mathcal{D}_q}(\theta)$ by (40);
- 7 Update $\theta \leftarrow \theta - \mu \nabla_{\theta} \mathcal{L}_{\mathcal{D}_q}(\theta)$;
- 8 **Return** θ

a ground truth KG, which is unavailable in the presence of Challenges C.2-C.4, we train the overall algorithm as a discriminative machine learning model. Accordingly, the RNN is encouraged to produce a KG that is useful in the sense that the resulting state estimator matches the corresponding label in the data-set (7). We leverage the differentiability of the GSP-EKF operation to enable gradient-based learning via backpropagation through time [71].

In particular, we set the loss measure to the ℓ_2 regularized MSE. For a labeled data-set $\mathcal{D}$ in (7), the loss is given by

$$\mathcal{L}_{\mathcal{D}}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{d=1}^{|\mathcal{D}|} \frac{1}{T_d} \sum_{t=1}^{T_d} \mathcal{L}_t^{(d)}(\theta) + \lambda \|\theta\|^2, \quad (40)$$

with $\lambda > 0$ being a regularization coefficient. The loss term for each time step t in the d th trajectory is obtained via

$$\mathcal{L}_t^{(d)}(\theta) = \left\| \hat{\mathbf{x}}_t^{(d)}(\theta) - \mathbf{x}_t^{(d)} \right\|^2, \quad (41)$$

with $\hat{\mathbf{x}}_t^{(d)}(\theta)$ being the t th estimator of GSP-KalmanNet with parameters θ applied to the d th trajectory. The resulting procedure optimizing θ based on (40) via mini-batch stochastic gradient descent (SGD) is summarized in Algorithm 3.

D. Discussion

The proposed GSP-KalmanNet is particularly tailored for tracking graph signals under the common realistic difficulties highlighted in Challenges C.1-C.4. It builds on the understanding that in order to simultaneously cope with incomplete domain knowledge and the inherent complexity associated with high-dimensional graph signals, one cannot rely on conventional EKF and its augmentations, but rather develop a dedicated machine learning architecture that arises from a graph signal-oriented tracking algorithm. For this reason, our first step in deriving GSP-KalmanNet is the development of GSP-EKF in Section III, followed by its conversion into a trainable algorithm that simultaneously benefits from the available (though possibly approximated) knowledge of the SS model alongside learning from data. The augmentation of the KG $\mathcal{K}_t$ computation by a DNN $\mathcal{K}_t(\theta)$ is not only a parametric graph filter as proposed in,

TABLE II
CHALLENGES AND COMPLEXITY COMPARISON

Challenge	EKF	GSP-EKF	KalmanNet	GSP-KalmanNet
C.1	X	V	X	V
C.2	X	X	X	V
C.3	X	X	V	V
C.4	X	X	V	V
Complexity	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$

e.g., [67], which is a linear mapping of the observations; it is in fact an abstract nonlinear mapping of indicative input features (innovation process, etc.) that would fit inside the GSP-EKF algorithm pipeline. This learned operation thus preserves the strengths of the designed GSP-EKF algorithm (Challenge C.1) and empowers it to operate over partially known dynamics and topologies (Challenges C.2-C.4).

Being derived from the GSP-EKF, GSP-KalmanNet shares its reduced computational complexity, which is a key component for scalability in light of Challenge C.1. The computational burden of applying a neural network is dictated by its number of weights. In GSP-KalmanNet, computing the Kalman Gain (having N input neurons and N output neurons) has a complexity of $\mathcal{O}(N^2)$, where KalmanNet involves $\mathcal{O}(N^3)$ computations. Regarding the classical algorithms, EKF is $\mathcal{O}(N^3)$, inverting an $N \times N$ matrix, while GSP-EKF has a complexity of $\mathcal{O}(N^2)$, transforming to the graph frequency domain by multiplying the signals with $N \times N$ matrix. The computational complexity is summarized in Table II. Nonetheless, in terms of inference time, as we show in Subsection V-E, GSP-KalmanNet is faster than KalmanNet, EKF, and GSP-EKF. This stems from the fact that the DNN-based methods are amenable to parallelization and acceleration of built-in software accelerators, e.g., PyTorch.

The ability of GSP-KalmanNet to cope with Challenges C.2-C.4 follows directly from its training method and its architecture. Specifically, GSP-KalmanNet is designed to directly produce the KG, bypassing the explicit tracking of the error covariance carried out by the EKF, which typically encapsulates the sensitivity of the model-based algorithm to inaccurate characterization of the underlying dynamics. The fact that training is evaluated based on the estimated state rather than the KG (which is the RNN output) allows GSP-KalmanNet to learn to compute a *surrogate* KG that enables coping with errors induced by mismatches in $\tilde{\mathbf{f}}_t(\cdot)$ and $\tilde{\mathbf{h}}_t(\cdot)$, and mispecified graph topology (Challenges C.2, C.4). While the error covariance is not explicitly tracked and is not accounted for in the loss (40), it can still be recovered from the KG in some settings [56]. In addition, as the KG is learned without explicit knowledge of the underlying stochasticity, GSP-KalmanNet can learn to track in non-Gaussian noise without knowing its distribution (Challenge C.3). These properties are empirically demonstrated in Section V. Table II details the challenges each benchmark in Section V is facing.

Preserving the model-based operation of the GSP-EKF, as done in GSP-KalmanNet, can potentially bring operational gains beyond increased accuracy and reduced complexity. For instance, it was shown in [55], [56] that in some settings one can

convert the KG into the state error covariance, providing useful uncertainty measures that are typically challenging to acquire in DL. Moreover, the fact that the prediction step of the EKF is preserved can be exploited to enable unsupervised learning, by evaluating GSP-KalmanNet during training based on its ability to predict the next observation rather than the next state [57]. In principle, one can consider alternative DNN-aided designs that are inspired by the operation of the EKF. For instance, one can utilize a DNN based on, e.g., multi-layered perceptrons or transformers to map the indicative features used in GSP-KalmanNet, directly to the hidden state $\mathbf{x}_t$, thus introducing additional abstractness while compromising some of the interpretability of GSP-KalmanNet and its aforementioned benefits. The detailed investigation of such alternatives is left for future work. Furthermore, the problem formulation in Subsection II-B assumes that the graph is stationary and has a known number of nodes. A candidate extension of GSP-KalmanNet is to deal with graphs with a varying number of nodes and structures, possibly by considering alternative architectures such as GNNs for computing the KG. We leave these investigations and extensions for future study.

V. EMPIRICAL STUDY

In this section, we experimentally evaluate GSP-KalmanNet¹ and compare its performance with those of the filtering model-based/data-driven benchmarks, detailed in the experimental setup in Subsection V-A. We consider cases with both full and partial domain knowledge. In the first study, presented in Subsection V-B, we consider two synthetic data-sets using nonlinear Gaussian SS models. In the second study in Subsection V-C we consider the practical application domain of power system monitoring [5], [6], [72] in case of both Gaussian and non-Gaussian noise. In Subsection V-D we numerically demonstrate some of the operational gains which arise from the interpretable design of GSP-KalmanNet. We conclude with complexity and latency evaluation in Subsection V-E.

A. Experimental Setup

We simulate dynamic systems that showcase the online tracking capabilities using both synthetic examples and real-world power grid trajectories. In our experiments, we consider both observations and hidden states over a known (though possibly inaccurate) graph structure with N nodes and adjacency matrix ($\mathbf{A}$ for an unweighted graph in Subsection V-B and $\mathbf{W}$ for a weighted graph in Subsection V-C). The observations are corrupted by two types of measurement noises: 1) An i.i.d. Gaussian noise, $\mathbf{v}_t$, with covariance matrix $\mathbf{R} = r^2 \cdot \mathbf{I}$; and 2) An exponential noise with different levels of parameter λ , used in the setting detailed in Subsection V-C. The evolution process noise is driven by i.i.d. Gaussian noise $\mathbf{e}_t$ with covariance matrix $\mathbf{Q} = q^2 \cdot \mathbf{I}$, setting a fixed ratio of $\log(q^2/r^2) = -20$ dB in the Gaussian case, and $\log(q^2/\lambda) = -20$ dB in the exponential case. In each experiment, we generate $D = 2,000$ trajectories

¹The source code and hyperparameters used are available at <https://github.com/NimrodLeinwand/GSP-KalmanNet>

of length $T = 200$, forming the data-set $\mathcal{D}$ in (7). This data-set is divided into train and validation, and is used by all data-driven methods for learning purposes. Additional 200 trajectories were generated as the test set, and were used to evaluate all the considered algorithms.

It is noted that the fact that the noise covariance matrices are diagonal indicates that $\mathbf{Q}$ and $\mathbf{R}$ are diagonalizable by the unitary $\mathbf{V}$, thereby satisfying Condition 1 of Theorem 1. However, Condition 2 is generally not fulfilled in our simulations. In particular, the state evolution functions used throughout our study are not separable in the graph frequency domain, while in Subsection V-C, $\mathbf{h}_t(\mathbf{L}, \mathbf{x})$ is also not separable in the graph frequency domain. Nevertheless, as can consistently be observed in the sequel, our reduced complexity GSP-based algorithms generally achieve similar MSE with their unconstrained and more complex counterparts, i.e., GSP-EKF generally achieves similar performance as the EKF, and our GSP-KalmanNet method is comparable to KalmanNet. This indicates that in the considered settings, we are not sacrificing performance by using the new scalable and reduced complexity GSP methods. These results can be attributed to the intrinsic graph structure embedded within the SS models.

We compare the *GSP-KalmanNet* with the following methods: (i) EKF (See Subsection II-C); (ii) *GSP-EKF* (See Subsection III-B); (iii) KalmanNet (See Subsection II-D); (iv) *GNN + RNN*: A DNN architecture that combines a GNN for spatial information capture with node-wise RNN for temporal correlation learning as in [46], [47]. This is a baseline data-based method without any assumption on the SS model.

B. Synthetic Data-Sets

We first evaluate GSP-KalmanNet on two synthetic dynamic systems, utilizing a range of linear and nonlinear functions. For each system, we generate a random unweighted graph using the DGL library [73], represented by the adjacency matrix $\mathbf{A}$.

1) *Nonlinear State Evolution Model*: We start our analysis by exploring a straightforward Gaussian SS model, with a state-evolution model that takes a nonlinear function with a sinusoidal form influenced by the underlying graph structure:

$$\mathbf{f}_t(\mathbf{A}, \mathbf{x}_t) = \sin(\mathbf{x}_t) + \cos(\mathbf{x}_t + \mathbf{A} \cdot \mathbf{x}_t), \quad (42)$$

where $\mathbf{x}_t + \mathbf{A} \cdot \mathbf{x}_t$ is the sum of neighboring nodes up to order 1. The measurement function is the following linear mapping: $\mathbf{h}_t(\mathbf{x}_t) = 3 \cdot \mathbf{x}_t$. Thus, according to (12), in this case, $\hat{\mathbf{H}}_t = 3 \cdot \mathbf{I}$ and $\hat{\mathbf{F}}_t = \frac{\partial \mathbf{f}}{\partial \mathbf{x}_t}$ can be computed from (42).

We analyze two scenarios: *full information*, where the SS model parameters (e.g., graph structure) match the one used for data generation, and *partial information*, with mismatched domain knowledge. The resulting MSEs are presented in Fig. 3(a) and 3(b) for $N = 10$ and $N = 30$, respectively, versus $\frac{1}{r^2}$, where r^2 is the measurement noise variance.

Full Information: In this setting, all algorithms have access to the dynamic and measurement functions, together with the underlying graph structure (represented by the adjacency matrix $\mathbf{A}$) employed during data generation. As evident from

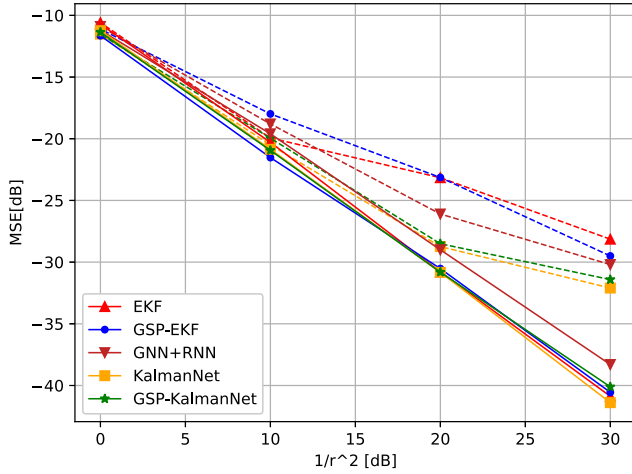
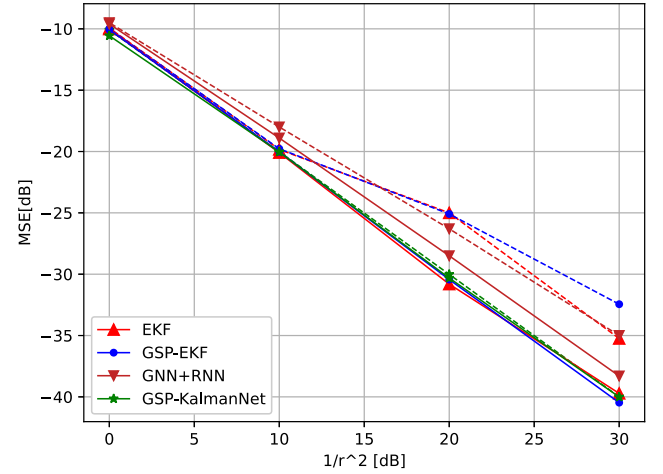
(a) Graph with $N = 10$ nodes.(b) Graph with $N = 30$ nodes.

Fig. 3. **Nonlinear state evolution model:** MSE versus inverse observation noise variance, $\frac{1}{r^2}$. The solid lines represent the benchmarks with full domain knowledge of the SS model, while the dashed lines are the case of partially known dynamics, given misspecified graph topology. The simulations were examined over random graphs with $N = 10$ and $N = 30$ nodes.

the solid lines in Fig. 3(a), both KalmanNet, and our proposed GSP-KalmanNet successfully learn the model-based mapping, achieving comparable outcomes, while GNN+RNN is slightly behind, not using domain knowledge.

Partial Information: Under partial model information, we evaluate the performance under a mismatch in the adjacency matrix, $\mathbf{A}$, having one edge randomly disconnected. Fig. 3(a) depicts the performance of the filtering methods in this case as the dashed lines. The trainable algorithms, KalmanNet, GNN+RNN, and GSP-KalmanNet, achieve better results compared to the classical methods across a wide range of noise levels. Notably, GSP-KalmanNet outperforms KalmanNet and GNN+RNN in terms of the training process, converges much faster and in a stable manner, which can be attributed to its lower parameter complexity.

Subsequently, we explored the same scenario on a larger unweighted graph with $N = 30$ nodes, each with 10 edges connected to the respective nodes. In this case, KalmanNet exhibits significantly higher RAM requirements for training, rendering it infeasible for training on the same hardware. In Fig. 3(b) it can be seen that GSP-KalmanNet outperforms the EKF, GSP-EKF, and data-driven GNN+RNN. Moreover, GSP-KalmanNet successfully learns an adapted KG to the setting of incomplete knowledge of the graph structure, achieving the same results as for the case of full domain knowledge. Our simulations (not shown here) present a big gap in the complexity and inference time, as shown in Subsection V-E.

2) *Nonlinear State Evolution and Measurement Models:* Concluding this section, we tackle a challenging nonlinear Gaussian SS model. Here, $\mathbf{x}_t$ and $\mathbf{y}_t$ represent graph signals of size $N = 9$ nodes, where each node is connected to 6 edges. The state evolution model adopts a sinusoidal form:

$$\mathbf{f}_t(\mathbf{x}_t) = \mathbf{x}_t + \sin(\mathbf{x}_t/10 + 3). \quad (43)$$

The measurement model, influenced by the graph frequency domain, is expressed as

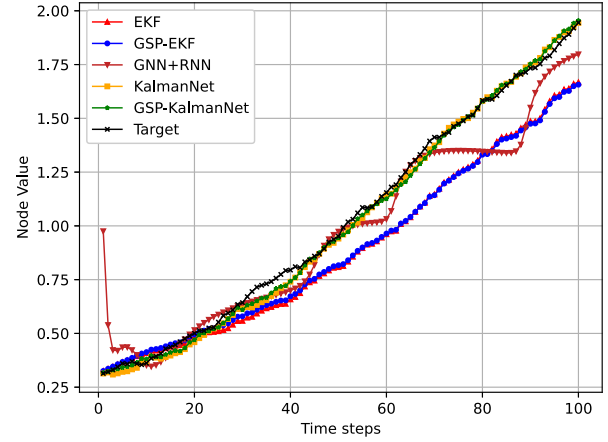
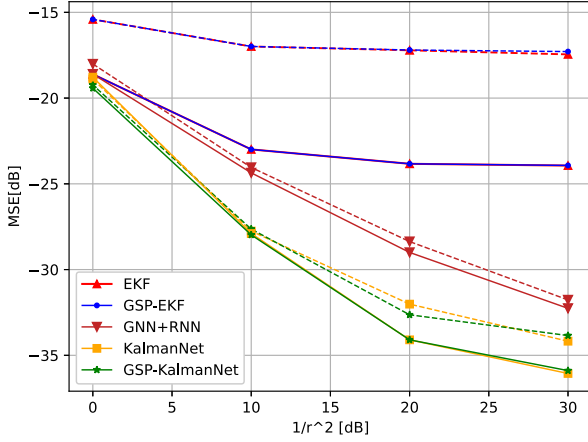
$$\mathbf{h}_t(\mathbf{V}, \mathbf{x}_t) = 0.5\mathbf{V}\mathbf{x}_t + 0.5(\mathbf{V}\mathbf{x}_t)^3, \quad (44)$$

where $(\cdot)^3$ is calculated elementwise. We again consider both *full information*, wherein the SS model parameters (in this case, both the functions $\mathbf{f}(\cdot)$, $\mathbf{h}(\cdot)$ and the graph structure that determines $\mathbf{V}$) align with those used for data generation, and *partial information*, wherein the domain knowledge deviates from the ground truth. The resulting MSE is displayed in Fig. 4(a).

Full Information: Under this setting, we observe in the solid lines in Fig. 4(a) a gap of more than 10dB between the DNN-aided methods (GNN+RNN, KalmanNet, and GSP-KalmanNet) and the model-based ones (EKF and GSP-EKF). This gap stems from the error induced in the local linearization of both the state evolution and the measurement functions, carried out by the model-based EKF and GSP-EKF. Among the data-driven methods, our GSP-KalmanNet achieves the best performance, both in terms of MSE, speed and stability in training, and convergence from almost any initial weights. Thus, compared to the black-box GNN+RNN, we see how using the state evolution and measurement functions empowers the performance.

An additional gain of GSP-KalmanNet is its ability not only to achieve improved error on average, but also to consistently obtain performance within close proximity of its averaged error. To show this property without cluttering Fig. 4(a), we report in Table III the resulting MSE values along with the confidence interval, computed via the standard deviation of the squared error. There, it is demonstrated that GSP-KalmanNet systematically achieves both best error, and smallest deviation.

Partial Information: In order to evaluate the performance of the different methods under partial model information, we assume in this scenario that we are not aware of 2 random edges in the graph. Furthermore, the state evolution function



(a) MSE versus different Gaussian measurement noise variance $\frac{1}{r^2}$. (b) State estimation at random node in the graph for a single trajectory.

Fig. 4. **Nonlinear state evolution and measurement functions:** MSE and instantaneous state estimate. The solid lines in (a) represent the benchmarks with full domain knowledge of the SS model, while the dashed lines are the case of partially known dynamics, given misspecified graph topology and mismatch in the given state evolution function $\mathbf{f}(\cdot)$.

TABLE III
VARIANCE [dB] OF EACH BENCHMARK IN DIFFERENT GAUSSIAN MEASUREMENT NOISE VARIANCE $\frac{1}{r^2}$ [dB]. THE VALUES CORRESPOND TO THE NONLINEAR STATE EVOLUTION AND MEASUREMENT FUNCTIONS DESCRIBED IN SUBSECTION V-B2

$\frac{1}{r^2}$ [dB]	0	10	20	30
EKF	-18 ± 0.081	-23.1 ± 0.032	-24.2 ± 0.019	-24.4 ± 0.015
GSP-EKF	-18.2 ± 0.077	-23.2 ± 0.026	-24.2 ± 0.021	-24.5 ± 0.011
GNN+RNN	-18.1 ± 0.091	-24.5 ± 0.047	-28.8 ± 0.031	-32.1 ± 0.014
KalmanNet	-18.8 ± 0.072	-27.3 ± 0.017	-34 ± 0.008	-36 ± 0.006
Ours	-18.9 ± 0.068	-27.6 ± 0.008	-34.1 ± 0.0006	-35.8 ± 0.0001

is characterized by an inaccurate rate (compared with the true model in (43)):

$$\mathbf{f}_t(\mathbf{x}_t) = \mathbf{x}_t + \sin(\mathbf{x}_t/9 + 3).$$

The resulting MSEs, illustrated by the dashed lines in Fig. 4(a), show that the data-driven GNN+RNN achieves a 15dB gap compared to the model-based methods, while the model-based DL methods, KalmanNet and GSP-KalmanNet, outperform with even more than 20dB. The source of this gap is mainly the model mismatch, which notably degrades purely model-based algorithms. In addition, all three data-driven methods demonstrate a narrower performance gap compared to their full-information counterparts. Regarding the training process, GSP-KalmanNet exhibits again much faster and more stable convergence than KalmanNet and GNN+RNN, which can be attributed to its reduced parameter complexity.

In addition, we illustrate in Fig. 4(b) the state estimation of a signal at a single node in the graph realization (taking 100 time steps of a smoother state-evolution function for ease of visualization). It can be seen that GSP-KalmanNet

systematically tracks the ground truth trajectory, compared to its counterparts that deviate from the trajectory. Moreover, the model-based algorithms, EKF and GSP-EKF, follow the dynamic trend (however, they drift slightly) compared to the GNN+RNN that trained only based on data, and curve around the desired trajectory.

C. Power Grid Monitoring - PSSE

In this subsection, we consider a practical application of power system monitoring [72], with the task of PSSE. A power system can be represented as an undirected weighted graph, $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathbf{W})$, where the set of nodes, $\mathcal{V}$, is the set of buses (generators/loads), and the edge set, $\mathcal{E}$, is the set of transmission lines between these buses [6], [62]. The weight matrix is usually based on the (nodal) admittance matrix of the considered system, which represents the complex weights over the transmission lines. In particular, in this work, we use the Laplacian matrix that is based on the susceptance matrix, $\mathbf{B}$ (see more in [74], [75]). The graph of $\mathbf{L}$ for the IEEE 14-bus system is illustrated in Fig. 6.

In a similar manner to the models in [11], [12], [13], we use the dynamical model in (5) to describe the state transition and the measurements, where the state vector, $\mathbf{x}_t$, consists of the voltage phases of the different buses and $\mathbf{y}_t$ contains the active powers at the buses. The commonly-used nonlinear AC power flow model [6], [76], which is based on Kirchhoff's and Ohm's laws, describes the relation between the complex power injection and voltage phasors. According to this model, the measurement function can be described by

$$[\mathbf{h}(\mathbf{x}_t)]_i = \sum_{j=1}^N ([\mathbf{G}]_{i,j} \cos([\mathbf{x}_t]_i - [\mathbf{x}_t]_j) + [\mathbf{B}]_{i,j} \sin([\mathbf{x}_t]_i - [\mathbf{x}_t]_j)), \quad (45)$$

$i = 1, \dots, N$, where $[\mathbf{G}]_{i,j}$ and $[\mathbf{B}]_{i,j}$ are the conductance and susceptance of the transmission line $(i, j) \in \mathcal{E}$ [76]. The goal

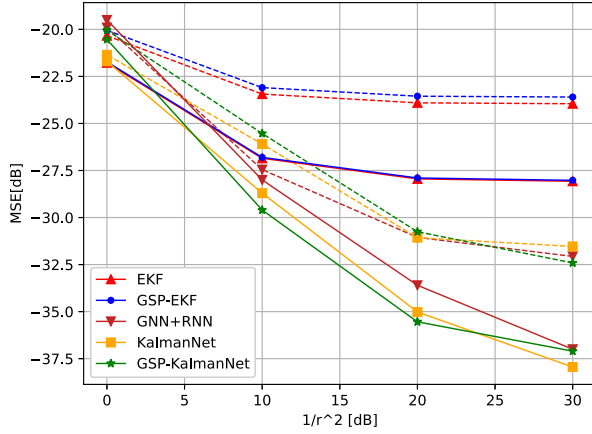
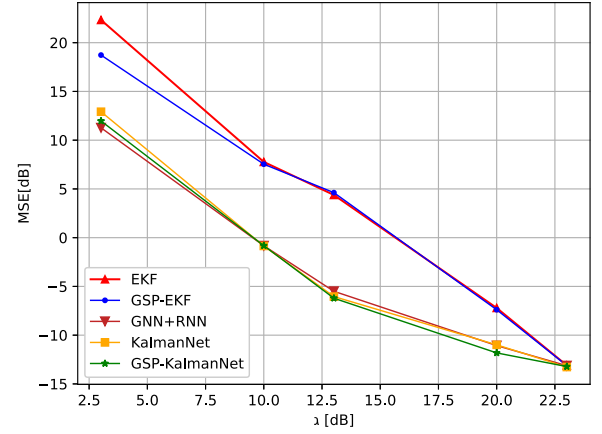
(a) Measurement noise $\mathbf{v}_t$ is Gaussian(b) Measurement noise $\mathbf{v}_t$ is exponential

Fig. 5. **Power grid data-set performance:** (a) represent the benchmarks MSE versus different Gaussian measurement noise variance $\frac{1}{r^2}$. The solid lines are full domain knowledge, while the dashed lines are the case of partially known dynamics, given misspecified graph topology. (b) represent the benchmarks MSE versus different exponential measurement noise with parameter λ , which assumed to be Gaussian, chosen through a grid search over the data.

of dynamic PSSE is to recover the state vector, $\mathbf{x}_t$, from the power measurements, $\mathbf{y}_t$, based on SS model as in (17) with the measurement function from (45). In this case, the (i, j) th element of $\hat{\mathbf{H}}_t = \frac{\partial \mathbf{h}}{\partial \mathbf{x}_t}$ is given by

$$[\mathbf{G}]_{i,j} \sin([\mathbf{x}_t]_i - [\mathbf{x}_t]_j) - [\mathbf{B}]_{i,j} \cos([\mathbf{x}_t]_i - [\mathbf{x}_t]_j), \quad i \neq j$$

and

$$\sum_{m=1}^N (-[\mathbf{G}]_{i,m} \sin([\mathbf{x}_t]_i - [\mathbf{x}_t]_m) + [\mathbf{B}]_{i,m} \cos([\mathbf{x}_t]_i - [\mathbf{x}_t]_m))$$

for $j = i$. In particular, since $[\mathbf{G}]_{i,j} = 0$ and $[\mathbf{B}]_{i,j} = 0$ for any $(i, j) \notin \mathcal{E}$, it can be seen that $[\hat{\mathbf{H}}_t]_{i,j} = 0$ for any $(i, j) \notin \mathcal{E}$. Thus, $\hat{\mathbf{H}}_t$ is a sparse matrix with the sparsity pattern of the graph. In the following, we examine two types of measurement noises: Gaussian and non-Gaussian.

1) *Gaussian Measurement Noise:* In the first setup, the observations are corrupted by i.i.d. Gaussian noise with different variance value, r^2 . We use the state-evolution function

$$\mathbf{f}_t(\mathbf{W}, \mathbf{x}_t) = 1 - (\mathbf{x}_t + \mathbf{W}\mathbf{x}_t), \quad (46)$$

which is influenced by the underlying weighted graph structure with $N = 14$ nodes presented in Fig. 6, where $\mathbf{x}_t + \mathbf{W}\mathbf{x}_t$ is the weighted sum of neighboring nodes up to order 1. We examine two scenarios: *full information*, wherein SS model parameters (the graph structure, depicted in $\mathbf{W}$) align with those used for data generation, and *partial information*, wherein $\mathbf{W}$ is inaccurate. The MSEs are shown in Fig. 5(a).

Full Information: In this context, we observe in the solid lines in Fig. 5(a) a gap of more than 10dB between the learnable methods and the classical ones, while GSP-KalmanNet outperforms both KalmanNet and the purely data-driven GNN+RNN. In this setting, we again needed to use numerous initialization weights for KalmanNet and GNN+RNN, while our GSP-KalmanNet trains in a stable manner for any initialization.

Partial Information: Assessing GSP-KalmanNet under partial model information entails having 10 edges randomly

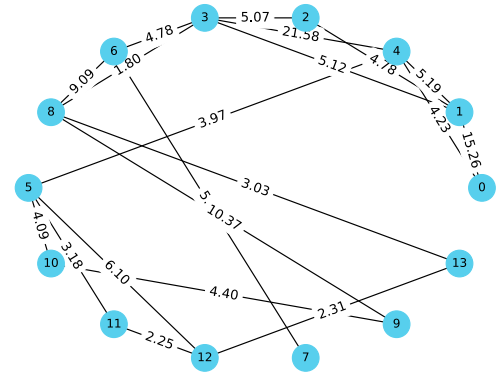


Fig. 6. Power grid graph with $N = 14$ nodes according to the IEEE 14-bus system.

disconnected, which affects the values in the assumed $\mathbf{W}$. The dashed lines in Fig. 5(a) illustrate that the trainable algorithms achieved an 8dB gap compared to the model-based methods.

2) *Exponential Measurement Noise:* In the second setting, the observations are corrupted by exponential noise with different levels of parameter λ , indicating the change over time on the noise variable, $\mathbf{v}_t$. We use the same measurement function (45) influenced by the same underlying weighted graph structure from Fig. 6. The state-evolution function in this case is a simple linear mapping

$$\mathbf{f}_t(\mathbf{x}_t) = \mathbf{x}_t + 0.05. \quad (47)$$

We examine the *full information* case, wherein SS model parameters (the graph structure, depicted in $\mathbf{G}$, $\mathbf{B}$, and the functions $\mathbf{f}(\cdot)$, $\mathbf{h}(\cdot)$) align with those used for data generation. However, the model-based algorithms were given a noise variance for both the state-evolution and measurement noises in the graph frequency domain, $\tilde{\mathbf{e}}_t$ and $\tilde{\mathbf{v}}_t$, assumed to be Gaussian, chosen through a grid search over the data. The MSEs are reported in Fig. 5(b). We can observe the performance

degradation of the model-based methods that have positive dB error for almost all λ values. A gap of more than 10dB of performance is achieved by the learnable methods, while GSP-KalmanNet outperforms both KalmanNet and the purely data-driven GNN+RNN. The rightest point shows that when dealing with almost no measurement noise, the data-driven methods learn the same mapping as the classical ones.

D. Gains of Interpretable Model-Based Deep-Learning Design

We next demonstrate the gains of having interpretable model-based deep-learning design. The fact that GSP-KalmanNet is obtained while preserving the operation of the GSP-EKF gives rise not only to a more understandable operation where one can assign meaning to internal features, but also to concrete operational gains. Here, we explore two of these gains: the ability to extract uncertainty in its estimate, and the capability to generalize from limited data.

1) *Extracting Uncertainty*: Providing a metric of uncertainty alongside a state estimate is often crucial when tracking a dynamical system. The model-based KF tracks the error covariance ($\hat{\Sigma}_t$) in (13), providing a time-dependent uncertainty measure from knowledge of the underlying statistics. However, deep learning based tracking systems struggle to reliably characterize uncertainty.

The interpretability of GSP-KalmanNet can be leveraged to estimate the error covariance, thus providing an uncertainty measure. In particular, when the observation noise covariance is available $\mathbf{R}$ (though the state noise $\mathbf{Q}$ is not required), the learned frequency domain KG $\tilde{\mathcal{K}}_t(\theta)$ computed by GSP-KalmanNet can be translated into an estimate of $\hat{\Sigma}_t$, as stated in the following lemma:

Lemma 2: Let $\tilde{\mathbf{R}}_t = \mathbf{V}^\top \mathbf{R} \mathbf{V}$ and $\tilde{\mathbf{H}}_t = \mathbf{V}^\top \hat{\mathbf{H}}_t \mathbf{V}$, and assume that $(\tilde{\mathbf{H}}_t^\top \cdot \tilde{\mathbf{H}}_t)^{-1}$ exists. Then, if $\tilde{\mathcal{K}}_t(\theta)$ is an accurate estimate of the KG, it holds that

$$\hat{\Sigma}_t = \tilde{\mathcal{K}}_t(\theta) \cdot \tilde{\mathbf{R}} \cdot \tilde{\mathbf{H}}_t (\tilde{\mathbf{H}}_t^\top \cdot \tilde{\mathbf{H}}_t)^{-1}, \quad (48)$$

is the graph frequency domain error covariance.

Proof: The lemma follows directly from [56], with adaptation to the graph frequency domain. ■

To showcase the usefulness of (48) in providing a faithful estimate of the error uncertainty, we next evaluate it in a numerical study. To that aim, we consider a SS model over a graph with $N = 10$ nodes, where $\mathbf{h}(\cdot)$ is a linear mapping given by $\hat{\mathbf{H}}_t = 10 \cdot \mathbf{I}$, while $\mathbf{f}(\mathbf{x}) = \cos(\mathbf{L}\mathbf{x})$ is the nonlinear sinusoidal state evolution function that includes the graph structure (via the Laplacian matrix $\mathbf{L}$).

In Table IV we compare the empirical MSE, averaged over 200 Monte Carol trails, to the corresponding error variance predictions, obtained by taking the trace of the estimated error covariances (computed via (48) for GSP-KalmanNet). Observing Table IV, we note that GSP-KalmanNet provides a reliable prediction of its empirical squared error, which approaches that produced by the model EKF that has full knowledge of the underlying SS model. It is noted that these estimates can

TABLE IV
EMPIRICAL VS. PREDICTED MSE COMPARISON

$\frac{1}{q^2} = \frac{1}{r^2}$ [dB]		10	20
EKF	Empirical MSE [dB]	−30	−40.03
	Predicted MSE [dB]	−30.1	−40.04
GSP-KalmanNet	Empirical MSE	−30	−40.01
	Predicted MSE	−26.99	−40.05

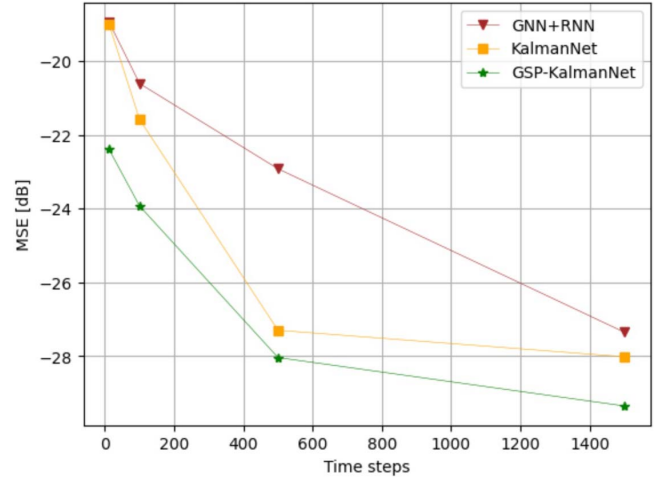


Fig. 7. Data-driven algorithms MSE performance of the nonlinear state evolution model vs different train-set sizes.

be made even more accurate by modifying the loss function to penalize the covariance estimates, see [56]. These findings indicate that the interpretable architecture of GSP-KalmanNet can be leveraged to provide operational gains in terms of uncertainty estimates.

2) *Smaller Train-Set Size*: Model-based deep learning designs are known to be data efficient [64], a trait that significantly enhances their utility in various applications. Unlike traditional data-driven algorithms, model-based deep learning approaches utilize principled suitable algorithmic steps, e.g., GSP-EKF, as a form of inductive bias for a machine learning architecture. We next showcase that this design indeed translates into improved data efficiency, i.e., the ability to generalize from limited data sets.

To that aim, we report in Fig. 7 data efficiency of GSP-KalmanNet compared to the other data-driven benchmarks KalmanNet and GNN+RNN for the Gaussian SS model scenario with nonlinear state evolution and measurement functions, detailed in Subsection V-B2. In Fig. 7 we compare the MSE performance, evaluated over 100 trajectories of length 200, for varying sizes of training data-sets, ranging from 20 to 1500 trajectories. Notably, GSP-KalmanNet exhibits a consistently accelerated convergence in addition to its improved performance, requiring only 500 trajectories to achieve MSE approached by the GNN+RNN architecture using at least 1,500 trajectories.

TABLE V
INFERENCE RUN TIME (IN SECONDS) COMPARISON BETWEEN THE
FILTERING METHODS OVER DIFFERENT SIZES OF GRAPHS. ‘-’ MARKS
INFEASIBLE COMPUTATION DUE TO LIMITED RAM

N	EKF	GSP-EKF	KalmanNet	GNN+RNN	Ours
10	2.7	2.6	2.5	2.23	1.9
50	3.5	3	2.7	2.4	2.1
300	-	18	-	30	3.8

E. Complexity and Latency

We conclude our numerical study by demonstrating that the performance benefits of GSP-KalmanNet are achieved with decreased computational complexity and latency compared with model-based EKF and the frequency adaption of GSP-EKF, as well as compared to its data-driven counterparts, KalmanNet and the GNN+RNN architecture. To that end, we provide, in addition to the complexity discussion in Subsection IV-D, an analysis of the average inference time computed over one trajectory. The run time of all tracking algorithms was evaluated on the same platform (Google Colab). For the large graphs, some of the benchmarks exhibited RAM issues.

The data is comprised of 100 different trajectories, with $T = 200$ time steps each. We used three different graph structures with sizes of $N = 10, 50, 300$. The resulting average run times are reported in Table V. As can be seen, the proposed GSP-KalmanNet, is significantly faster than the benchmark algorithms, especially regarding large graphs. It stems from the fact that the EKF needs to compute the Jacobians and matrix inversions at each time instance to produce its KG (Complexity of $\mathcal{O}(N^3)$). Even though GSP-EKF has less complexity (transforming to the graph frequency domain by multiplying with $N \times N$ matrix - ($\mathcal{O}(N^2)$)). GSP-EKF is still slower than GSP-KalmanNet, due to the parallelization and acceleration of built-in software accelerators, e.g., PyTorch. GSP-KalmanNet is faster than GNN+RNN since it has fewer weights and it does not crash as KalmanNet, having a complexity of $\mathcal{O}(N^3)$. These results, combined with the MSE performance detailed in the previous subsections, demonstrate the ability of GSP-KalmanNet in learning to track graph signals while coping with Challenges C.1-C.4.

VI. CONCLUSION

We proposed a hybrid model-based/data-driven algorithm that tracks graph signals in complex scenarios. Our novel algorithm, GSP-KalmanNet, leverages the principles of the EKF while harnessing the graph structure using GSP techniques to simplify the analysis in the graph frequency domain. Furthermore, our method incorporates DL components to handle partial and approximate modeling. Our empirical findings demonstrate that GSP-KalmanNet outperforms both model-based and data-driven techniques in terms of accuracy, robustness, and run time performance for tracking high-dimensional graph signals, while coping with the challenges of complexity and partially known dynamics.

APPENDIX PROOF OF THEOREM 1

Since $\tilde{\mathbf{H}}_t = \mathbf{V}^\top \mathbf{H}_t \mathbf{V}$, by using (12), the GFT definition in (2), and derivative rules, one obtains

$$\begin{aligned} \tilde{\mathbf{H}}_t &= \mathbf{V}^\top \mathbf{H}_t \mathbf{V} = \mathbf{V}^\top \nabla_{\mathbf{x}} \mathbf{h}_t(\mathbf{L}, \mathbf{x})|_{\mathbf{x}=\hat{\mathbf{x}}_{t|t-1}} \mathbf{V} \\ &= \nabla_{\tilde{\mathbf{x}}} \tilde{\mathbf{h}}_t(\mathbf{L}, \mathbf{V}\tilde{\mathbf{x}})|_{\tilde{\mathbf{x}}=\hat{\tilde{\mathbf{x}}}_{t|t-1}} \mathbf{V}, \end{aligned} \quad (49)$$

where $\tilde{\mathbf{h}}_t(\mathbf{L}, \mathbf{V}\tilde{\mathbf{x}}) = \mathbf{V}^\top \mathbf{h}_t(\mathbf{L}, \mathbf{x})$. By substituting (36b) in (49), we obtain that under Condition 2, the r.h.s. of (49) is a diagonal matrix and thus, $\tilde{\mathbf{H}}_t$ from the l.h.s. of (49) is also diagonal. Similarly, (36a) implies that $\tilde{\mathbf{F}}_t$ is also a diagonal matrix. From Condition 1, $\tilde{\mathbf{Q}}$ is a diagonal matrix. Thus, the state covariance estimate of $\tilde{\Sigma}_{t|t-1}$ in (33) is a sum of diagonal matrices for $t = 1$. This implies that $\tilde{\Sigma}_{1|0}$ is a diagonal matrix. Similarly, based on Condition 1 and (11b), $\tilde{\Sigma}_{1|0}$ is a diagonal matrix. Now, one can observe that the KG in the graph frequency domain from (20), for $t = 1$, is also diagonal. Therefore, $\tilde{\mathcal{K}}_1 = \tilde{\mathcal{K}}_1^{(\text{GSP})}$, where $\tilde{\mathcal{K}}_t^{(\text{GSP})}$ is defined in (30). By substituting these results in (21), we obtain that $\tilde{\Sigma}_{1|1}$ is diagonal. Thus, it can be similarly shown by induction that under Conditions 1-2, $\tilde{\Sigma}_{t|t}$ and $\tilde{\Sigma}_{t|t-1}$ are diagonal matrices, meaning

$$\tilde{\mathcal{K}}_t = \tilde{\mathcal{K}}_t^{(\text{GSP})}, \quad \forall t \in \mathbb{Z}.$$

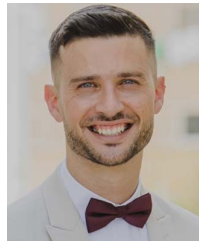
Thus, the GSP-EKF in Algorithm 1 coincides with the EKF in the graph frequency domain, detailed in Subsection III-A, which have the same MSE as the standard EKF presented in Subsection II-C. Hence, under Conditions 1-2, the GSP-EKF coincides with the EKF.

REFERENCES

- [1] G. Sagi, N. Shlezinger, and T. Routtenberg, “Extended Kalman filter for graph signals in nonlinear dynamic systems,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.
- [2] A. Ortega, P. Frossard, J. Kovačević, J. M. F. Moura, and P. Vandergheynst, “Graph signal processing: Overview, challenges, and applications,” *Proc. IEEE*, vol. 106, no. 5, pp. 808–828, May 2018.
- [3] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, “The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains,” *IEEE Signal Process. Mag.*, vol. 30, no. 3, pp. 83–98, May 2013.
- [4] R. Ramakrishna and A. Scaglione, “Grid-graph signal processing (grid-GSP): A graph signal processing framework for the power grid,” *IEEE Trans. Signal Process.*, vol. 69, pp. 2725–2739, 2021.
- [5] Y. Zhao, J. Chen, and H. V. Poor, “A learning-to-infer method for real-time power grid multi-line outage identification,” *IEEE Trans. Smart Grid*, vol. 11, no. 1, pp. 555–564, Jan. 2020.
- [6] G. B. Giannakis, V. Kekatos, N. Gatsis, S.-J. Kim, H. Zhu, and B. F. Wollenberg, “Monitoring and optimization for power grids: A signal processing perspective,” *IEEE Signal Process. Mag.*, vol. 30, no. 5, pp. 107–128, Sep. 2013.
- [7] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.
- [8] R. E. Larson, R. M. Dressler, and R. S. Ratner, “Application of the extended Kalman filter to ballistic trajectory estimation.” Stanford Research Inst. Menlo Park, CA, Tech. Rep., 1967. [Online]. Available: <https://apps.dtic.mil/sti/citations/AD0875083>
- [9] E. A. Wan and R. Van Der Merwe, “The unscented Kalman filter,” in *Kalman Filtering and Neural Networks*, S. Haykin, Ed., New York, NY, USA: Wiley, 2001, pp. 221–280.
- [10] J. Durbin and S. J. Koopman, *Time Series Analysis by State Space Methods*, vol. 38., Oxford, U.K.: OUP Oxford, 2012.

- [11] J. Zhao, M. Netto, and L. Mili, "A robust iterated extended Kalman filter for power system dynamic state estimation," *IEEE Trans. Power Syst.*, vol. 32, no. 4, pp. 3205–3216, Jul. 2017.
- [12] C. Carquex, C. Rosenberg, and K. Bhattacharya, "State estimation in power distribution systems based on ensemble Kalman filtering," *IEEE Trans. Power Syst.*, vol. 33, no. 6, pp. 6600–6610, Nov. 2018.
- [13] A. K. Singh and B. C. Pal, "Decentralized dynamic state estimation in power systems using unscented transformation," *IEEE Trans. Power Syst.*, vol. 29, no. 2, pp. 794–804, Mar. 2014.
- [14] A. Anis, A. Gadde, and A. Ortega, "Towards a sampling theorem for signals on arbitrary graphs," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2014, pp. 3864–3868.
- [15] A. G. Marques, S. Segarra, G. Leus, and A. Ribeiro, "Sampling of graph signals with successive local aggregations," *IEEE Trans. Signal Process.*, vol. 64, no. 7, pp. 1832–1843, Apr. 2016.
- [16] Y. Tanaka, "Spectral domain sampling of graph signals," *IEEE Trans. Signal Process.*, vol. 66, no. 14, pp. 3752–3767, Jul. 2018.
- [17] T. Routtenberg, "Non-Bayesian estimation framework for signal recovery on graphs," *IEEE Trans. Signal Process.*, vol. 69, pp. 1169–1184, 2021.
- [18] A. Kroizer, T. Routtenberg, and Y. C. Eldar, "Bayesian estimation of graph signals," *IEEE Trans. Signal Process.*, vol. 70, pp. 2207–2223, 2022.
- [19] A. Amar and T. Routtenberg, "Widely-linear MMSE estimation of complex-valued graph signals," *IEEE Trans. Signal Process.*, vol. 71, pp. 1770–1785, 2023.
- [20] G. Sagi and T. Routtenberg, "MAP estimation of graph signals," *IEEE Trans. Signal Process.*, vol. 72, pp. 463–479, 2024.
- [21] P. Li, N. Shlezinger, H. Zhang, B. Wang, and Y. C. Eldar, "Graph signal compression by joint quantization and sampling," *IEEE Trans. Signal Process.*, vol. 70, pp. 4512–4527, 2022.
- [22] H. E. Egilmez, E. Pavez, and A. Ortega, "Graph learning from data under Laplacian and structural constraints," *IEEE J. Sel. Topics Signal Process.*, vol. 11, no. 6, pp. 825–841, Sep. 2017.
- [23] L. Shi, "Kalman filtering over graphs: Theory and applications," *IEEE Trans. Autom. Control*, vol. 54, no. 9, pp. 2230–2234, Sep. 2009.
- [24] D. Romero, V. N. Ioannidis, and G. B. Giannakis, "Kernel-based reconstruction of space-time functions on dynamic graphs," *IEEE J. Sel. Topics Signal Process.*, vol. 11, no. 6, pp. 856–869, Sep. 2017.
- [25] A. Soule, K. Salamati, A. Nucci, and N. Taft, "Traffic matrix tracking using Kalman filters," *Perform. Eval. Rev.*, vol. 33, pp. 24–31, Dec. 2005.
- [26] E. Isufi, G. Leus, and P. Banelli, "2-dimensional finite impulse response graph-temporal filters," in *Proc. IEEE Global Conf. Signal Inf. Process. (GlobalSIP)*, 2016, pp. 405–409.
- [27] E. Isufi, A. Loukas, N. Perraudin, and G. Leus, "Forecasting time series with VARMA recursions on graphs," *IEEE Trans. Signal Process.*, vol. 67, no. 18, pp. 4870–4885, Sep. 2019.
- [28] E. Isufi, P. Banelli, P. D. Lorenzo, and G. Leus, "Observing and tracking bandlimited graph processes from sampled measurements," *Signal Process.*, vol. 177, Dec. 2020, Art. no. 107749.
- [29] L. Su, Z. Han, J. Zhao, and W. Wang, "Graph-frequency domain Kalman filtering for industrial pipe networks subject to measurement outliers," *IEEE Trans. Ind. Informat.*, vol. 20, no. 5, pp. 7977–7985, May 2024.
- [30] P. Di Lorenzo, P. Banelli, E. Isufi, S. Barbarossa, and G. Leus, "Adaptive graph signal processing: Algorithms and optimal sampling strategies," *IEEE Trans. Signal Process.*, vol. 66, no. 13, pp. 3584–3598, Jul. 2018.
- [31] B. Cox and V. Elvira, "Sparse Bayesian estimation of parameters in linear-Gaussian state-space models," *IEEE Trans. Signal Process.*, vol. 71, pp. 1922–1937, 2023.
- [32] E. Chouzenoux and V. Elvira, "Sparse graphical linear dynamical systems," 2023, *arXiv:2307.03210*.
- [33] V. Elvira and E. Chouzenoux, "Graphical inference in linear-Gaussian state-space models," *IEEE Trans. Signal Process.*, vol. 70, pp. 4757–4771, 2022.
- [34] J. Gu, X. Yang, S. De Mello, and J. Kautz, "Dynamic facial analysis: From Bayesian filtering to recurrent neural network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1548–1557.
- [35] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
- [36] S. S. Rangapuram, M. W. Seeger, J. Gasthaus, L. Stella, Y. Wang, and T. Januschowski, "Deep state space models for time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1–10.
- [37] B. Millidge, A. Tschantz, A. Seth, and C. Buckley, "Neural Kalman filtering," 2021, *arXiv:2102.10021*.
- [38] S. Jouaber, S. Bonnabel, S. Velasco-Forero, and M. Pilte, "NNAKF: A neural network adapted Kalman filter for target tracking," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2021, pp. 4075–4079.
- [39] P. Becker, H. Pandya, G. Gebhardt, C. Zhao, C. J. Taylor, and G. Neumann, "Recurrent Kalman networks: Factorized inference in high-dimensional deep feature spaces," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 544–552.
- [40] A. Klushyn, R. Kurl, M. Soelch, B. Cseke, and P. van der Smagt, "Latent matters: Learning deep state-space models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 10234–10245, 2021.
- [41] S. R. Jondhale and R. S. Deshpande, "Kalman filtering framework-based real time target tracking in wireless sensor networks using generalized regression neural networks," *IEEE Sensors J.*, vol. 19, no. 1, pp. 224–233, Jan. 2019.
- [42] W. Jiang and J. Luo, "Graph neural network for traffic forecasting: A survey," *Expert Systems with Applications*, vol. 207, 2022, Art. no. 117921.
- [43] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, Apr. 2020.
- [44] F. Gama, A. G. Marques, G. Leus, and A. Ribeiro, "Convolutional neural network architectures for signals supported on graphs," *IEEE Trans. Signal Process.*, vol. 67, no. 4, pp. 1034–1049, Feb. 2019.
- [45] A. Parada-Mayorga, L. Ruiz, and A. Ribeiro, "Graphon pooling in graph neural networks," in *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 2021, pp. 860–864.
- [46] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 4, 2020, pp. 5363–5370.
- [47] J. Skarding, B. Gabrys, and K. Musial, "Foundations and modeling of dynamic networks using dynamic graph neural networks: A survey," *IEEE Access*, vol. 9, pp. 79143–79168, 2021.
- [48] H. He, C.-K. Wen, S. Jin, and G. Y. Li, "Model-driven deep learning for MIMO detection," *IEEE Trans. Signal Process.*, vol. 68, pp. 1702–1715, 2020.
- [49] H. K. Aggarwal, M. P. Mani, and M. Jacob, "MoDL: Model-based deep learning architecture for inverse problems," *IEEE Trans. Med. Imag.*, vol. 38, no. 2, pp. 394–405, Feb. 2019.
- [50] N. Shlezinger, Y. C. Eldar, and S. P. Boyd, "Model-based deep learning: On the intersection of deep learning and optimization," *IEEE Access*, vol. 10, pp. 115384–115398, 2022.
- [51] N. Shlezinger and Y. C. Eldar, "Model-based deep learning," *Found. Trends® Signal Process.*, vol. 17, no. 4, pp. 291–416, 2023.
- [52] M. Nagahama, K. Yamada, Y. Tanaka, S. H. Chan, and Y. C. Eldar, "Graph signal restoration using nested deep algorithm unrolling," *IEEE Trans. Signal Process.*, vol. 70, pp. 3296–3311, 2022.
- [53] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. Van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.
- [54] G. Revach, X. Ni, N. Shlezinger, R. J. van Sloun, and Y. C. Eldar, "RTSNet: Learning to smooth in partially known state-space models," *IEEE Trans. Signal Process.*, vol. 71, pp. 4441–4456, 2023.
- [55] I. Klein, G. Revach, N. Shlezinger, J. E. Mehr, R. J. G. van Sloun, and Y. C. Eldar, "Uncertainty in data-driven Kalman filtering for partially known state-space models," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2022, pp. 3194–3198.
- [56] Y. Dahan, G. Revach, J. Dunik, and N. Shlezinger, "Uncertainty quantification in deep learning based Kalman filters," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2024, pp. 13121–13125.
- [57] G. Revach, N. Shlezinger, T. Locher, X. Ni, R. J. van Sloun, and Y. C. Eldar, "Unsupervised learned Kalman filtering," in *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 2022, pp. 1571–1575.
- [58] I. Buchnik, D. Steger, G. Revach, R. J. van Sloun, T. Routtenberg, and N. Shlezinger, "Latent-KalmanNet: Learned Kalman filtering for tracking from high-dimensional signals," *IEEE Trans. Signal Process.*, vol. 72, pp. 352–367, 2024.
- [59] N. Shlezinger and T. Routtenberg, "Discriminative and generative learning for linear estimation of random signals [lecture notes]," *IEEE Signal Process. Mag.*, vol. 40, no. 6, pp. 75–82, Sep. 2023.
- [60] A. Sandryhaila and J. M. F. Moura, "Discrete signal processing on graphs," *IEEE Trans. Signal Process.*, vol. 61, no. 7, pp. 1644–1656, Apr. 2013.
- [61] A. Ortega, *Introduction to Graph Signal Processing*. Cambridge, U.K.: Cambridge Univ. Press, 2022, ch. 3.
- [62] E. Drayer and T. Routtenberg, "Detection of false data injection attacks in smart grids based on graph signal processing," *IEEE Syst. J.*, vol. 14, no. 2, pp. 1886–1896, Jun. 2020.

- [63] L. Dabush and T. Routtenberg, "Verifying the smoothness of graph signals: A graph signal processing approach," *IEEE Trans. Signal Process.*, 2024, doi:10.1109/TSP.2024.3439554.
- [64] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, "Model-based deep learning," *Proc. IEEE*, vol. 111, no. 5, pp. 465–499, May 2023.
- [65] M. Gruber, "An approach to target tracking," MIT Lexington Lincoln Lab, Tech. Rep., 1967. [Online]. Available: <https://apps.dtic.mil/sti/citations/tr/AD0654272>
- [66] H. Coskun, F. Achilles, R. DiPietro, N. Navab, and F. Tombari, "Long short-term memory Kalman filters: Recurrent neural estimators for pose regularization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5524–5532.
- [67] J. Liu, E. Isufi, and G. Leus, "Filter design for autoregressive moving average graph filters," *IEEE Trans. Signal Inf. Process. Netw.*, vol. 5, no. 1, pp. 47–60, May 2019.
- [68] D. I. Shuman, P. Vandergheynst, and P. Frossard, "Chebyshev polynomial approximation for distributed signal processing," in *Proc. Int. Conf. Distrib. Comput. Sensor Syst. Workshops*, 2011, pp. 1–8.
- [69] L. Dieci and T. Eiroola, "On smooth decompositions of matrices," *SIAM J. Matrix Anal. Appl.*, vol. 20, no. 3, pp. 800–819, 1999.
- [70] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. NIPS Workshop Deep Learn.*, 2014.
- [71] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, Oct. 1990.
- [72] S. Kanna, D. H. Dini, Y. Xia, S. Y. Hui, and D. P. Mandic, "Distributed widely linear Kalman filtering for frequency estimation in power networks," *IEEE Trans. Signal Inf. Process. Netw.*, vol. 1, no. 1, pp. 45–57, Mar. 2015.
- [73] M. Wang et al., "Deep graph library: A graph-centric, highly-performant package for graph neural networks," 2019, *arXiv:1909.01315*.
- [74] L. Dabush, A. Kroizer, and T. Routtenberg, "State estimation in partially observable power systems via graph signal processing tools," *Sensors*, vol. 23, no. 3, 2023, Art. no. 1387.
- [75] M. Halihal, T. Routtenberg, and H. V. Poor, "Estimation of complex-valued Laplacian matrices for topology identification in power systems," *IEEE Trans. Signal Process.*, early access, 2024, doi:10.1109/TSP.2024.3436524.
- [76] A. Abur and A. Gomez-Exposito, *Power System State Estimation: Theory and Implementation*. New York, NY, USA: Marcel Dekker, 2004.



Itay Buchnik received the B.Sc. and M.Sc. degrees, both *cum laude*, from the School of Electrical and Computer Engineering, Ben-Gurion University, Israel, in 2021 and 2023, respectively, specializing in machine learning for signal processing, model-based deep learning, computer vision, and graph signal processing. He was honored with the JNF Scholarship for "Israel 2040" in 2021, the IDSI Grant for "Access and Usage of Cloud Resources" in 2022, and the Kaufman Award for outstanding research achievements in 2023. He was also the

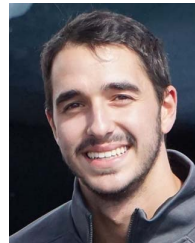
recipient of the Best Student Presentation Award at the 2022 EURASIP Summer School on "Data and Graph Driven Learning for Communications and Signal Processing." During his academic career, he interned with the ELTA Systems, the Forschungszentrum Jülich Lab in Germany, and Persimio. He is currently working as an AI Researcher with the Elbit Systems.



Guy Sagi received the B.Sc. (*summa cum laude*) degree in electrical and computer engineering from Ben-Gurion University of the Negev, Israel, in 2022, specializing in signal processing and communication. He is currently working as an Embedded Software Engineer. His research interests include machine learning for signal processing, model-based deep learning, and graph signal processing.



Nimrod Leinwand received the B.Sc. degree in electrical and computer engineering from Ben-Gurion University, Israel, in 2023, specializing in signal processing and computers. He is an Electronics Engineer with the Soreq Nuclear Research Center.



Yuval Loya received the B.Sc. degree from the School of Electrical and Computer Engineering, Ben-Gurion University, Israel, in 2023, specializing in computers and signal processing. During his academic career, he interned with Apple. He is currently working as a SoC Design Verification Engineer with Apple.



Nir Shlezinger (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in electrical and computer engineering from Ben-Gurion University, Israel, in 2011, 2013, and 2017, respectively. He is an Assistant Professor with the School of Electrical and Computer Engineering, Ben-Gurion University, Israel. From 2017 to 2019, he was a Postdoctoral Researcher with the Technion, and from 2019 to 2020, he was a Postdoctoral Researcher with Weizmann Institute of Science. His research interests include communications, information theory, signal processing, and machine learning. He was awarded the FGS Prize for his research achievements. He is the recipient of the 2024 IEEE ComSoc Fred W. Ellersick Award and the 2024 Krill Prize for outstanding young researchers.



Tirza Routtenberg (Senior Member, IEEE) received the B.Sc. degree from the Technion Israel Institute of Technology, Haifa, Israel, in 2005, and the M.Sc. and Ph.D. degrees in electrical and computer engineering from Ben-Gurion University, Israel, in 2007 and 2012, respectively. She is an Associate Professor with the School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Israel. She was a Postdoctoral Fellow with the School of Electrical and Computer Engineering, Cornell University, in 2012–2014. Since

2014, she has been a Faculty Member with the School of Electrical and Computer Engineering, Ben-Gurion University. From 2022 to 2023, she was a William R. Kenan, Jr. Visiting Professor for distinguished teaching with Princeton University. Her research interests include signal processing in smart grids, statistical signal processing, estimation and detection theory, and signal processing on graphs. She was an Associate Editor of IEEE TRANSACTIONS ON SIGNAL AND INFORMATION PROCESSING OVER NETWORKS (2019–2023), an Associate Editor of IEEE SIGNAL PROCESSING LETTERS (since 2021), and a Subject Editor of *Signal Processing* Elsevier (since 2024). She was the recipient of the Best Student Paper Award at the International Conference on Acoustics, Speech and Signal Processing (ICASSP) 2011, IEEE International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP) 2013 (co-author), ICASSP 2017 (co-author), and IEEE Workshop on Statistical Signal Processing (SSP) 2018 (co-author). She was awarded the Negev Scholarship in 2008, the Lev-Zion scholarship in 2010, the Marc Rich Foundation Prize in 2011, and the Toronto Prize for Excellence in Research in 2021.